

CHAPTER 4

Client-Side Programming The JavaScript™ Language

Author : Mrs. M. Anbu

This chapter will introduce you to the JavaScript™ programming language, which is supported by nearly all traditional web browsers in use today. Our focus in this chapter will be on the study of JavaScript as a programming language, largely independently of how it might relate to a web browser. In the next chapter we will cover in more detail how JavaScript programs can interact with documents contained in a web browser by, for example, changing the style or even the content of a document. Therefore, this chapter will not include a section on the case study blogging application, which we defer to the next chapter.

While you may know that JavaScript is often used for relatively simple tasks within browsers, you should not be misled into thinking that it is therefore a language of limited power. To the contrary, as we will learn, it is in several respects a particularly powerful language, and in fact incorporates some programming language concepts that may be new to you. So, in addition to being useful for web programming, studying JavaScript may broaden your understanding of programming languages in general.

4.1 History and Versions of JavaScript

JavaScript was initially developed by Brendan Eich as part of the Netscape 2.0 release. The language was called LiveScript for a while in its early stages. But on December 4, 1995, before the final release of Netscape 2.0, the language was publicly announced as JavaScript. This name change was apparently intended to link the scripting language with the rising popularity of Sun's Java programming language, but has caused a fair amount of confusion over the years. Although there are strong similarities between the core syntaxes of JavaScript and Java (and C++, for that matter), and although JavaScript was designed to interact with Java applets, there are also tremendous differences between JavaScript and Java.

JavaScript became especially popular after the 1.1 release as part of Netscape's 3.0 browser. This version of JavaScript allowed web page developers to produce the familiar rollover effect: as the mouse moves over images, the images change (we'll learn how to produce this effect in Chapter 5). Other JavaScript versions followed as part of later Netscape browser releases. Meanwhile, Microsoft introduced the JScript[®] programming language (the JavaScript name was owned by Sun) in July of 1996, and this language similarly went through a series of revisions as new versions of Internet Explorer were released.

To confuse matters somewhat more, there is yet another flavor of JavaScript, known as ECMAScript. Soon after announcing JavaScript, Sun and Netscape approached an organization then known as the European Computer Manufacturers Association (ECMA)

to produce a standard for JavaScript. The third edition of the ECMA-262 standard [ECMA-262] was released in 1999 and has been a significant help in bringing JavaScript and JScript closer together. JavaScript 1.5, the version implemented in Mozilla 1.4, conforms with ECMAScript Edition 3 (barring bugs), and JScript versions 5.5 (the version implemented in IE5.5) and beyond also appear to be compliant. Other browsers, such as Opera and Safari, also support the ECMAScript standard.

I will therefore focus on ECMAScript Edition 3 in this chapter, and refer to this simply as “JavaScript.” All of the examples presented run in both Mozilla 1.4 and IE6, except as noted.

4.2 Introduction to JavaScript

I’d like to begin with a “Hello World!” JavaScript program. There’s just one problem: JavaScript itself has no statement for performing output. Instead, the JavaScript language specification leaves it up to browsers to supply output (and input) methods. Happily, certain methods are supported by almost all modern browsers, even though technically they are not part of any standard. So we’ll use one of these methods, `alert()`, to write a JavaScript “Hello World!” program (the `window` object is discussed in Section 4.3.1):

```
window.alert("Hello World!");
```

We can execute this program by referencing a file containing it from a `script` element within an HTML document. For example, if the JavaScript code given is placed within a file named `JSHelloWorld.js` in the same directory as the HTML document of Figure 4.1, then loading this document into a browser will cause the code to be executed. If this document is loaded into Mozilla 1.4, then the pop-up window shown in Figure 4.2 will appear. The browser window will be unresponsive until this *alert box* is dismissed, either by clicking the OK button or by closing the window (in Windows, as shown here, by clicking the $\times$ icon in the upper right corner of the alert box window). The client area of the browser window itself will be completely empty after the alert box is dismissed.

```
<!DOCTYPE html
  PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>
      JSHelloWorld.html
    </title>
    <script type="text/javascript" src="JSHelloWorld.js">
    </script>
  </head>
  <body>
  </body>
</html>
```

FIGURE 4.1 HTML document that loads and executes the JavaScript program in file `JSHelloWorld.js`.



FIGURE 4.2 Alert box generated by a JavaScript statement.

As with output, there is no input statement in JavaScript itself. Again, though, most browsers implement a `prompt()` method that can be called as illustrated by the following code:

```
var inString = window.prompt("Enter JavaScript code to be tested:",  
                             "");
```

This pops up a window that displays the value of its first string argument and also provides a text box in which a user can enter information and that initially contains the string given by the second argument (Figure 4.3). The value returned by the `prompt()` method is the string entered by the user, assuming that the user clicks OK after entering the string (fuller details of `prompt()` are provided in Chapter 5).

Now that you've been introduced to JavaScript, let's briefly consider some of the ways in which developing JavaScript programs can differ from more traditional (Java/C++) program development before delving into details of the language.

4.3 JavaScript in Perspective

This section provides an overview of how JavaScript relates to various categories of programming languages and how this impacts development of JavaScript programs. We'll also look briefly at a few JavaScript software development tools so that you can experiment with your own code as you read the rest of the chapter.

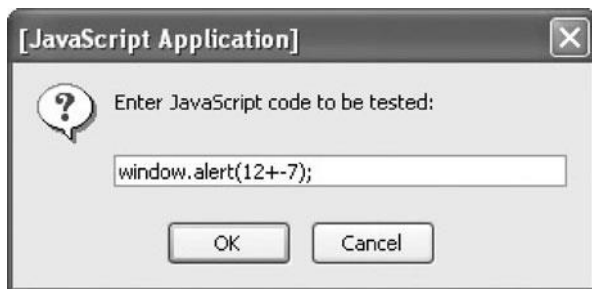


FIGURE 4.3 Prompt pop-up window with user input entered.

4.3.1 Scripting Languages

As you probably noticed, we did not need to compile the “Hello World!” JavaScript program before executing it. Programming languages that do not need to be compiled before execution are known as *interpreted* languages. Software that reads and executes a program written in an interpreted language is known as an *interpreter*. Most modern browsers contain a JavaScript interpreter.

Programs written in interpreted languages are generally easier to maintain than programs written in compiled languages such as Java. For example, there’s no need to remember to recompile interpreted programs after modifying them, and there are fewer files to manage, since there are no compiled versions of source files. Programs written in an interpreted language are also often simpler than equivalent programs written in a traditional compiled language. The one-line “Hello World!” program is one example of this. As another example, consider that while Java has a wide variety of numeric data types (`int`, `float`, `double`, etc.), JavaScript has a single `Number` data type. So some of the Java complexities of choosing an appropriate numeric data type and casting between types are not present in JavaScript.

There are, of course, some advantages to compiled languages. First, interpreted programs generally take longer to execute than similar programs written in compiled languages. One reason for this is that something like compilation has to occur during execution of an interpreted language, whereas it occurs before execution in a compiled language. Similarly, the simplicity of an interpreted language such as JavaScript comes at the cost of less efficient code. For example, the reason that Java provides many different numeric data types is so that the compiler can choose an implementation for an arithmetic operation that is optimized for the problem at hand. JavaScript, on the other hand, uses a generic—and therefore generally less than optimal—implementation for every arithmetic operation.

Many interpreted languages are, like JavaScript, also scripting languages. A *scripting language* is a specialized programming language designed to be used to automate tasks within a particular software environment. For example, if you have used Linux, you may have heard of *shell scripts*. These are generally small programs that issue Linux shell commands (the sorts of commands you type in within a shell prompt window) in order to cause the operating system to perform certain functions. For instance, shell scripting is a convenient way to change the file extension on all of the files within a directory. This could be done by hand, but scripting can be used to automate what might otherwise be a tedious process.

JavaScript is also intended to run within a certain software environment, or more precisely, within many software environments. The original LiveScript language was intended to be used as part of Netscape’s LiveWire™ web server software as well as within Netscape browsers. JScript can similarly be run within Microsoft’s IIS web server as well as within Internet Explorer browsers. In addition, many Microsoft operating systems also provide a Script Host application that essentially allows the Windows equivalent of Linux shell scripting to be performed using JScript. Certain other software applications also contain JavaScript interpreters, such as the Dreamweaver web-development tool.

To accommodate this diversity of script environments, each JavaScript implementation effectively consists of two software components. The primary component is a *scripting engine*, which includes a JavaScript interpreter as well as core ECMAScript functionality that must be present regardless of the script environment. The JavaScript scripting engine is

the focus of this chapter. The second component of a complete JavaScript implementation is a *hosting environment*, which provides environment-specific capabilities to JavaScript programs running within the environment. The `alert()` and `prompt()` methods are part of the hosting environment of Mozilla and IE6. These and many other hosting environment features found in most modern browsers will be covered in Chapter 5.

All of the hosting environment capabilities as well as many scripting engine capabilities are provided through objects. Objects that are required by the ECMAScript definition and therefore provided by the scripting engine are known as *native* objects, and those provided by the hosting environment are known as *host* objects. In the JavaScript “Hello World!” example, `window` is the name of a native object (even though some of its methods are provided by the hosting environment). A *built-in* object is a native object that is automatically constructed during scripting engine initialization rather than being constructed as a result of a call to a constructor during program execution. The `window` object is an example of a built-in object, since it can be used immediately without constructing it explicitly.

4.3.2 Writing and Testing JavaScript Programs

JavaScript code can be written using various tools. Many JavaScript programs are small; for these, a simple text editor such as Windows Notepad is probably sufficient. If you use the GNU[®] Emacs editor (which runs on multiple platforms), it will colorize your JavaScript code if you include the following statements in the `.emacs` initialization file:

```
;; Turn on C++/other highlighting
(global-font-lock-mode)

;; Turn on a javascript mode for .js files
(require 'generic-x)
(add-to-list 'generic-extras-enable-list 'javascript-generic-mode)
```

For larger projects, commercial tools tailored to JavaScript development are available. For example, Microsoft’s Visual InterDev[®] Web development system includes various JavaScript productivity features such as automatically providing lists of object properties and methods.

Once you’ve entered your JavaScript code, the next step is to execute your program, since JavaScript is not compiled and therefore does not require a compilation step before execution. While the lack of compilation streamlines the development process, it also means that there are no compiler error messages to alert you to potential problems before you execute a JavaScript program. Instead, any error messages are produced when you execute the program. What’s more, to keep the language simple and flexible, JavaScript forgoes many safety features—commonly found in compiled languages—that help programmers to avoid hard-to-debug errors. For instance, it is not necessary to declare variables in JavaScript, which might seem like a programmer-friendly feature at first. But as an illustration of the problems this can cause, note that if you misspell the variable name on the left-hand side of a JavaScript assignment statement, then the scripting engine will silently create a new variable with the misspelled name. Obviously, you’ll want to keep these sorts of problems in mind when you are debugging your JavaScript code.

As illustrated in Figure 4.1, executing a browser-based JavaScript program requires opening an HTML document in your browser. You can write your own documents to run each of your programs, or you can edit an existing document such as the one in Figure 4.1 so that it loads the file containing your program rather than `jsHelloWorld.js`. A simpler alternative using software available from the textbook Web site is described in the next section.

If you run your program and it does not behave as expected, the first step is to check for any browser-generated error messages. The details of this step of course depend on the browser you're using for your tests. If you have installed a full version of Mozilla as described in Appendix A, then error messages appear in its JavaScript console. You can open this console by selecting **Tools|WebDevelopment|JavaScript Console** from the browser menu. This console will display any syntax errors or run-time exceptions encountered (the code implementing Mozilla's features sometimes throws exceptions to this window, so I suggest opening the console and clicking its Clear button before running your JavaScript code). A nice feature of the Mozilla JavaScript console is that its error messages normally contain a hyperlink that, if clicked, will open a window displaying your JavaScript code and highlighting the offending line. This window is not an editor, though, so you'll still need to go back to your development software in order to make changes to your code.

IE6's response to JavaScript errors depends on the browser options you have selected as well as on whether or not certain other software is available on your system. A JavaScript problem is often indicated by the presence of a small yellow exclamation-mark icon in the lower left corner of the browser window (on the status bar). Clicking this icon will pop up a dialog box describing the error. However, instead of relying on this icon, I suggest selecting **Tools|Internet Options . . .** from the IE6 menu and click on the Advanced tab in the pop-up window that appears. Then check the "Display a notification about every script error" checkbox. The browser should then automatically pop up the error dialog box whenever a JavaScript syntax or run-time problem is encountered. Note that if you have debugging software installed and debugging enabled (also controlled via **Internet Options . . .**), instead of seeing this error window you will be asked whether or not you want to begin a debugging session (see Section 4.13 about JavaScript debuggers).

The Details section of the IE6 error window tells the type of error and its location (file name, line within the file, and character within the line). If multiple errors are encountered, the pop-up window will contain information about one of the errors and a Next button that will allow you to step through other error messages.

If your browser does not produce any error messages but your program does not behave as expected, you'll need to locate the source of the problem. The same sorts of bug location techniques that apply to other languages apply to JavaScript. Some common techniques are to desk check (simulate by hand) your program, add output statements so that you can see the results of intermediate computations, or run an interactive debugger. You'll need to understand certain JavaScript concepts in order to use an interactive debugger effectively, so this approach is covered in Section 4.13.

After locating the source of errors and changing your code, you can retest your program by reloading the HTML document containing the code into your browser. If your browser is currently accessing this document, this is as simple as clicking the Reload (Mozilla) or Refresh (IE6) button in the browser toolbar.

4.4 Basic Syntax

Much of JavaScript's basic syntax is identical to that of Java or C++. To illustrate these similarities, consider the JavaScript program of Figure 4.4, which plays the high-low guessing game with the user. That is, the program randomly picks a whole number between 1 and 1000, and the user attempts to guess the number. After each guess, the user is either told that the guess was correct or is told whether the guessed number is too high or too low. This continues until the user guesses the computer's number.

Let's begin by considering some ways in which JavaScript syntax is similar to Java and C++. First, comments in JavaScript are the same as those in Java: `//` begins a comment that ends at the end of the line containing the comment, and `/*` and `*/` may be used to contain multiline comments (not shown in this example). Also, JavaScript's use of braces to enclose statement blocks and its syntax for assignment statements and control constructs such as `if` and `while` should be familiar. Another similarity is that semicolons (`;`) can be used at the end of JavaScript statements as they are in Java and C++. Technically, most JavaScript statements do not require this, but for compatibility with compiled languages and to avoid possible hard-to-locate errors, I suggest always using semicolons just as you would in Java.

JavaScript uses the familiar dot notation to represent properties and methods of objects. So we can infer, for instance, that `random()` is a method of the `Math` object (which like `window` is a built-in object). This method returns a randomly chosen random

```
// HighLow.js

var thinkingOf; // Number the computer has chosen (1 through 1000)
var guess;      // User's latest guess

// Initialize the computer's number
thinkingOf = Math.ceil(Math.random()*1000);

// Play until user guesses the number
guess = window.prompt("I'm thinking of a number between 1 and 1000." +
    " what is it?", "");
while (guess != thinkingOf)
{
    // Evaluate the user's guess
    if (guess < thinkingOf) {
        guess = window.prompt("Your guess of " + guess +
            " was too low. Guess again.", "");
    }
    else {
        guess = window.prompt("Your guess of " + guess +
            " was too high. Guess again.", "");
    }
}

// Game over; congratulate the user
window.alert(guess+" is correct!");
```

FIGURE 4.4 JavaScript high-low program.

decimal number from 0 up to but not including 1, and `Math.ceil()` returns the smallest integer greater than or equal to its argument. The net effect of applying these methods in the program is that a random integer between 1 and 1000 is assigned to `thinkingof`.

Some other similarities between Java and JavaScript that will be covered in more detail later in this chapter include the following. First, variable names in JavaScript are case sensitive, and the rules for choosing variable names are similar to those used in C++ and Java (although JavaScript of course has a somewhat different collection of reserved words that cannot be used as variable names). The syntax for argument passing in method calls and for array access (not shown in the example program in Figure 4.4) is also similar to Java's.

In addition, most of the strings used to represent various logical, string, and numeric operators, such as `!=`, `+`, and `*`, are identical to those used in Java and C++. (Recall that *operators* in a programming language appear within expressions, where an *expression* is a portion of a program that can be *evaluated* in order to obtain a value. For example, the expression `1+2` contains the operator `+` and, when evaluated, returns the value 3. The `+` operator has two *operands*, 1 and 2.)

While JavaScript thus has many similarities with Java and C++, there are also some significant differences. First, there is no `main()` function or method. Instead, the JavaScript scripting engine simply begins evaluating the first line of code it reads. Second, the variable declarations (statements beginning with the keyword `var`) do not specify a data type for the variables declared. Typed variables are another safety feature that is missing in JavaScript: if you assign a string value to a variable that you were intending to treat as a numeric variable, JavaScript will silently assign the string value to the variable. On the other hand, this can be viewed as adding to JavaScript's flexibility: if you want to (I don't recommend this), you can deliberately store a `Number` value in a variable at one time in a program's execution and at a later time store a `String` value in the same variable. In computer science terminology, JavaScript is a *dynamically typed* language whereas Java and C++ are *statically typed*.

This brings us to a third significant difference: JavaScript performs many conversions between data types automatically. In this program, the `prompt()` method returns a string value, so the values assigned to `guess` are all strings. But the value of `thinkingof` is a number. In Java, an attempt to apply a relational operator such as `<` or `!=` to compare `String` and `int` operands would result in a compile error. But JavaScript automatically converts the string argument to a number so that the comparison can be performed. We'll learn more about JavaScript's type conversion rules later. For now, note that this automatic conversion feature, while convenient, comes at the expense of run-time computation required to detect the need for conversion and to choose an appropriate conversion to apply.

The JavaScript program of Figure 4.4—as well as every other example in this chapter that begins with a comment consisting of a file name—is part of the chapter 4 directory of the download package available from the textbook supplements Web site (the URL is given in the Preface). The download package also includes an HTML document named `testjs.html` that you can use to load and execute this chapter's examples. Simply open the `testjs.html` file in your browser; in Mozilla, you can do this by selecting **File|Open File . . .** from the menu and navigating to the `testjs.html` file in the file selector window that appears. When the file loads, the browser's client area will be blank, but a pop-up prompt box (as in Figure 4.3) will appear. Enter the name of the example file (such as `highLow.js`) in this prompt box, and click OK. Your browser will then load the example file and execute the JavaScript code that it contains.

Now that you have some familiarity with the basic syntax of JavaScript, the next several sections will explore how JavaScript handles specific programming issues, including variables and data types, statements and operators, objects, arrays, and functions (methods).

4.5 Variables and Data Types

As previously mentioned, variables do not have data types in JavaScript. However, every variable has a value, and every value belongs to one of six JavaScript data types. Every numeric value is of type Number, string values are of type String, the literals `true` and `false` represent the two Boolean values, the literal `null` represents the one value of type Null, and every object is of type Object. The remaining type, Undefined, is the type of the value represented by any variable that has been declared but has not yet been assigned a value. Attempts to use such variables in Java will be flagged as errors by the Java compiler, but it is up to the programmer to avoid this situation in JavaScript. The five JavaScript data types other than Object are sometimes referred to as *primitive data types*.

In JavaScript, `typeof` is an operator that provides information about the data type of a value stored in a variable, as shown in Table 4.1. For example, the JavaScript code

```
// typeof.js
var i;
var j;
j = "Not a number";
alert("i is " + (typeof i) + "\n" +
      "j is " + (typeof j));
```

produces the output of Figure 4.5. A common use of the `typeof` operator is to test that a variable has been defined before attempting to use it. Notice, though, that `typeof` returns the string `undefined` for several different reasons, not only when a variable contains the value of type Undefined.

As mentioned previously, JavaScript can perform automatic conversions between data types. Tables 4.2, 4.3, and 4.4 show rules used to convert to Boolean, String, and Number types, respectively. NaN (“not a number”) and Infinity, mentioned in these tables,

TABLE 4.1 Values Returned by `typeof` for Various Operands

Operand Value	String <code>typeof</code> Returns
<code>null</code>	<code>object</code>
Boolean	<code>boolean</code>
Number	<code>number</code>
String	<code>string</code>
Native Object representing function	<code>function</code>
Native Object not representing function	<code>object</code>
Declared variable with no value	<code>undefined</code>
Undeclared variable	<code>undefined</code>
Nonexistent property of an Object	<code>undefined</code>



FIGURE 4.5 Result of executing `TypeOf.js`.

TABLE 4.2 Data Type Conversions to Boolean

Original Value	Value as Boolean
Undefined	false
null	false
0	false
NaN	false
"" (empty string)	false
Any other value	true

TABLE 4.3 Data Type Conversions to String

Original Value	Value as String
Undefined	undefined
null	null
true, false	true, false
NaN	NaN
Infinity, -Infinity	Infinity, -Infinity
Other Number up to ≈ 20 digits	Integer or decimal representation
Number over ≈ 20 digits	Scientific notation
Object	Call to <code>toString()</code> method on the object

TABLE 4.4 Data Type Conversions to Number

Original Value	Value as Number
Undefined	NaN
null, false, "" (empty string)	0
true	1
String representing number	Represented number
Other String	NaN
Object	Call to <code>valueOf()</code> method on the object

Client-Side Programming

are special Number values that can occur if, for example, you divide by 0 (0/0 produces NaN while any positive number divided by 0 produces Infinity). `toString()` (Table 4.3) and `valueOf()` (Table 4.4) are methods that are defined (either implicitly or explicitly) for every JavaScript object. We'll have more to say about these methods for some of JavaScript's built-in objects later in this chapter.

We learned earlier that, when comparing a String and a Number value with a comparison operator such as `<`, JavaScript will automatically convert the String value to a Number before performing the comparison. Some other common situations in which the scripting engine performs automatic type conversion are:

- The condition portion of statements such as `if` and `while` is automatically converted to Boolean.
- The value of the accessor portion of an array access (for example, the 3 in an expression such as `records[3]`) is automatically converted to String.

Additional type-conversion scenarios are covered in Section 4.7.

The strings used to name variables are called *identifiers*. Any string that begins with a letter or underscore (`_`), consists only of these characters and digits, and is not one of the reserved words listed in Figure 4.6 is a valid identifier in JavaScript. (The JavaScript specification also allows certain other characters in identifiers; see Section 7.6 of the ECMAScript specification [ECMA-262] for full details.) As mentioned previously, identifiers are case sensitive.

Finally, if a JavaScript program assigns a value to a variable without first declaring the variable (with a `var` statement), then the scripting engine will automatically create the variable. So, for example, the following is a legal JavaScript program that will create a variable, assign it a value, and then display the value in an alert box:

```
testing = "Does this work?";  
window.alert(testing);
```

However, I strongly recommend using `var` to declare your variables. First, using `var` consistently to create variables makes it easier to determine all of the variables used in your program, which should facilitate future program maintenance. Moreover, you can avoid

abstract	boolean	break	byte	case	catch
char	class	const	continue	debugger	default
delete	do	double	else	enum	export
extends	false	final	finally	float	for
function	goto	if	implements	import	in
instanceof	int	interface	long	native	new
null	package	private	protected	public	return
short	static	super	switch	synchronized	
this	throw	throws	transient	true	try
typeof	var	void	volatile	while	with

FIGURE 4.6 JavaScript reserved words.

certain subtle errors if you use `var` rather than assignments to create all variables (but I'll leave the details to the exercises).

4.6 Statements

JavaScript has three types of statements. First, there is the *expression statement*, that is, a statement that consists entirely of an expression. Typical expression statements include assignment and increment statements, such as

```
i = 5;
j++;
```

The second type is the *block statement*, which is simply a set of statements enclosed in braces `{` and `}`. Finally, there are statements that begin with keywords such as `var` and `if`, which are (of course) called *keyword statements*.

We have already met the `var` statement, which is JavaScript's one statement for declaring a variable. Syntactically, the `var` keyword is followed by a variable declaration list, which is a comma-separated list of identifiers. Each identifier in a variable declaration list may optionally be followed by an equals sign (`=`) and then an arbitrary expression that acts as an initializer. So the JavaScript code

```
var i, msg="hi", o=null;
```

declares three variables named `i`, `msg`, and `o`, assigning them values of types `Undefined`, `String`, and `Null`, respectively.

Table 4.5 lists the JavaScript keyword statements that have direct analogs in Java and C++. I assume that you're already familiar with these statements as they're used in Java, so I won't cover them in detail here. Instead, Figure 4.7 illustrates the use of several of

TABLE 4.5 Some JavaScript Keyword Statements

Statement Name	Syntax
If-then	<code>if (expr) stmt</code>
If-then-else	<code>if (expr) stmt else stmt</code>
Do	<code>do stmt while (expr)</code>
While	<code>while (expr) stmt</code>
For	<code>for (part1 ; part2 ; part3) stmt</code>
Continue	<code>continue</code>
Break	<code>break</code>
Return-void	<code>return</code>
Return-value	<code>return expr</code>
Switch	<code>switch (expr) { cases }</code>
Try	<code>try try-block catch-part</code>
Throw	<code>throw expr</code>

these statements. This program will successively display three alert boxes. The first two boxes will contain the message A JavaScript exception can be anything, and the third will contain `i = 3`. Comments in the code point out small differences between some of the JavaScript keyword statements and their Java analogs.

4.7 Operators

In this section, we'll cover many of JavaScript's operators. Since most of these operators are similar to operators, in Java and C++, our focus will not be on the basic functionality of the operators, but instead on some specific ways in which JavaScript operators differ from those in Java.

Before covering the operators themselves, let's review some terminology related to operators. A *binary operator* is one such as `*` that has two operands. Other operators, such as `!` (the logical NOT operator) and `typeof`, have a single operand and are called *unary operators*. A unary operator may be either *prefix*, meaning that it precedes its operand, or *postfix*, meaning that it follows.

JavaScript also has one *ternary operator* that takes three operands: the *conditional operator*, which separates its first and second operands with a question mark (?) and its second and third operands with a colon (:). Although this operator also appears in Java and C++, you may not have used it before, so I'll describe it briefly here. The conditional operator provides a convenient means to code if-then-else logic within a single statement: the first operand (preceding the question mark) is treated as a Boolean value, and the overall

```
// KeywordStmts.js

// Can use 'var' to define a loop variable inside a 'for'
for (var i=1; i<=3; i++) {

    switch (i) {

        // 'case' value can be any expression and data type,
        // not just constant int as in Java. Automatic
        // type conversion is performed if needed.
        case 1.0 + 2:
            window.alert("i="+ i);
            break;
        default:
            try {
                throw("A JavaScript exception can be anything");
                window.alert("This is not executed.");
            }
            // Do not supply exception data type in 'catch'
            catch (e) {
                window.alert("Caught:"+ e);
            }
            break;
    }
}
```

FIGURE 4.7 JavaScript program illustrating several keyword statements.

expression evaluates to either the value of the second operand, if the Boolean value is `true`, or the value of the third operand otherwise. For example,

```
window.alert( "Error " + (debugLevel>2 ? details : "" ) );
```

adds a string representing the value of the `details` variable to an output message if the relational expression `debugLevel>2` evaluates to `true`. Otherwise, the empty string is appended to the message.

4.7.1 Precedence

Table 4.6 lists the JavaScript operators covered in this chapter, in order of their operator precedence. Recall that, in an unparenthesized expression, an operator of higher *precedence* is performed before one of lower precedence. Table 4.6 lists operators from highest to lowest precedence. As in most languages, parentheses can be used to override these default precedences, with innermost parenthesized subexpressions evaluated before those farther out.

The assignment, conditional, and prefix unary operators are *right associative*, which means that if there is a tie among multiple operators at the same precedence level, the rightmost operator is applied first. For example, an expression such as

```
a *= b += c
```

is equivalent to

```
a *= (b += c)
```

TABLE 4.6 Precedence (High to Low) for Selected JavaScript Operators

Operator Category	Operators
Object creation	<code>new</code>
Postfix unary	<code>++</code> , <code>--</code>
Prefix unary	<code>delete</code> , <code>typeof</code> , <code>++</code> , <code>--</code> , <code>+</code> , <code>-</code> , <code>~</code> , <code>!</code>
Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>
Additive	<code>+</code> , <code>-</code>
Shift	<code><<</code> , <code>>></code> , <code>>>></code>
Relational	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
(In)equality	<code>==</code> , <code>!=</code> , <code>===</code> , <code>!==</code>
Bitwise AND	<code>&</code>
Bitwise XOR	<code>^</code>
Bitwise OR	<code> </code>
Logical AND	<code>&&</code>
Logical OR	<code> </code>
Conditional and assignment	<code>?:</code> , <code>=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>+=</code> , <code>-=</code> , <code><<=</code> , <code>>>=</code> , <code>>>>=</code> , <code>&=</code> , <code>^=</code> , <code> =</code>

The postfix unary operators cannot be applied repeatedly. For example,

```
i++ ++
```

is not a syntactically correct expression. Therefore, associativity does not apply to these operators. The remaining operators—those that are not in the conditional, assignment, or unary categories—are *left associative*, so ties among these operators are broken in favor of the leftmost operator.

Almost all of the operator symbols in Table 4.6 should already be familiar to you, either from your knowledge of Java or C++ or from material earlier in this chapter (two operators that are not found in Java or C++, `===` and `!==`, are covered in some detail later). However, many of these symbols have a somewhat different meaning in JavaScript than they do in Java or C++. We'll give an overview of the primary differences in the remainder of this section.

4.7.2 Type Conversion

One of the key differences between JavaScript and Java/C++ operators has to do with automatic type conversion. Let's begin with the additive and multiplicative operators `+`, `-`, `*`, `/`, and `%`. Each of these operators will convert its operands to `Number` (according to the rules of Table 4.4) if they are not already of this type. Thus, for example, the following is legal JavaScript code and outputs 3:

```
window.alert("5" - 2);
```

One exception to this conversion rule involves the `+` operator: much as in Java, if either or both of the operands of the `+` operator are of type `String`, then the non-`String` operand (if any) will automatically be converted to `String` and the two `String` operand values will be concatenated. (Use of the `+` operator in this way was illustrated in the calls to the `alert()` method within the high-low game program of Figure 4.4.)

Similarly, we learned earlier that if a `String` and a `Number` value are compared with a relational operator such as `<`, then the `String` will be converted to a `Number` before the comparison is performed. More generally, for each of the four relational operators listed in Table 4.6, unless both of its operands are of type `String`, the operands are converted to `Number` as needed before the comparison is performed. (Although a bit off the topic of conversions, note that relational expressions involving the special Infinity `Number` value generally behave as you would expect. For example, everything except Infinity is less than Infinity. The special NaN value, on the other hand, is treated as if it is not less than, equal to, or greater than any other value, including itself.)

If both operands to one of the relational operators or one of the operators `==` and `!=` are of type `String`, then a lexicographic string comparison is performed. Recall that a *lexicographic comparison* of two strings `s1` and `s2` is performed as follows. String `s1` is lexicographically less than string `s2` if, at the first character position where the strings differ, the character value for `s1` is numerically less than the corresponding character value for `s2`. If the strings do not differ in any character position, then `s1` is lexicographically less than `s2` if the length of `s1` is less than the length of `s2`. If neither of these conditions holds, then the strings are lexicographically equal.

As you're probably aware, lexicographic comparisons of `String` objects in Java cannot be performed with relational or equality operators, but instead are normally performed with calls to methods such as `equals()` and `compareTo()`. For example, if `str1` and `str2` are two Java `String` objects, then to compare these strings lexicographically we would normally use code such as

```
if (str1.equals(str2)) { ... }
```

On the other hand, if `str3` and `str4` are JavaScript variables containing `String` values, JavaScript's automatic type conversion feature allows us to instead write

```
if (str3 == str4) { ... }
```

The type conversion rules for the `==` and `!=` operators generally follow the pattern we have seen earlier: if the operands are not both of type `String`, then both are typically converted to `Number` for comparison purposes. However, there are several exceptions. First, the value of the `Undefined` type (the type of a declared variable that has not been assigned a value) is considered equal to `null`, the one value of the `Null` type. Also, if one of the operands is an `Object` value and this object is an instance of the `Date` built-in object (described in Section 4.12.3), then the `Object` is converted to `String` instead of `Number`. Finally, if one of the operands is an `Object` value representing a host object rather than a native object, then the type conversion of the object is implementation dependent.

To allow for (in)equality comparisons without type conversion, JavaScript provides the two *strict* operators. The strict equality (`===`) operator evaluates to `true` only if both operands are of the same type (without conversion) and have exactly the same value. The strict inequality (`!==`) operator is of course just the logical NOT of strict equality. For values of type `Object`, the values are “the same” only if both operands are references to exactly the same object. If you're not quite sure what an object reference is, this concept is covered in detail in Section 4.10.4. For now, it's enough to know that this is how Java defines its `==` operator when applied to two Java objects (and in fact the JavaScript `==` operator also behaves this way when applied to two `Object` values).

Certain other operators perform natural automatic conversions. For example, each of the unary `+` and `-` operators converts its operand to `Number` (this is in fact all that the unary `+` operation does). Likewise, the logical AND, logical OR, and logical negation (`!`) operators will all convert their operands to `Boolean` as needed. (Technically, the logical AND and OR operators behave somewhat differently than this. But if you only use them in the normal way—within the condition expression of statements such as an `if` or `while` statement—then they will effectively behave as just described.)

4.7.3 Bit Operators

A JavaScript programmer cannot access the internal bit representation of a `Number` value. However, JavaScript does supply the same bit-level operators found in Java: bitwise (ones complement) `NOT(~)`; the bitwise AND, XOR, and OR operators; and the shift operators (Table 4.6). The JavaScript operators first convert their operands to 32-bit twos-complement integers by converting them to `Number` (Table 4.4) if needed, then truncating decimal values

Client-Side Programming

to integers, and finally retaining only the low-order 32 bits for integers that exceed 32 bits. The operators are then applied as if they were operating within a machine with 32-bit registers, which implies, for example, that the result of a left shift operator `<<` will be truncated to 32 bits if necessary. After the operator processing is completed, the 32-bit result is converted back to a Number value. For purposes of this final conversion, the 32-bit result is treated as an unsigned 32-bit integer by the unsigned right shift (`>>>`) operator and as a 32-bit twos-complement integer by the other operators.

As an example of using bit operators in JavaScript, consider the program

```
// ComplementOne.js
window.alert(~1);
```

This program will output `-2`, because `~1` evaluates to a string of 31 1 bits followed by a 0 bit, which represents `-2` when treated as a signed twos-complement 32-bit integer for purposes of conversion to Number. On the other hand,

```
// ComplementOne.js (part 2)
window.alert((~1) >>> 0);
```

outputs `4294967294` ($2^{32} - 2$), which is the value of the same 32-bit string treated as an unsigned integer for purposes of conversion to Number.

4.8 Literals

The most basic elements of any programming language are its *literals*, that is, the strings of characters that directly represent values in the language. We've already noted that `null` is a JavaScript literal representing the (one) value of the Null data type, and that `true` and `false` are the literals representing the two Boolean values. This section provides some syntactic details about JavaScript number and string literals.

The JavaScript number literals are much like those in Java. For example, numbers can be written as integers (whole numbers) or decimals, and in either of these forms scientific notation can be used to indicate a power of 10. For example, `-12.4e12` represents the value -12.4×10^{12} . Java hexadecimal notation, e.g., `0xfa0`, is also a valid form of number literal. Every JavaScript number literal represents a value belonging to the JavaScript Number data type. Since JavaScript has only one type of number, a Java literal that represents a number of a specified type (such as `27.3f`, which represents a floating point value) is not a valid number literal in JavaScript. All Number values are represented using a 64-bit floating point format that provides approximately 16 decimal digits of precision for both integer and decimal values and can represent numbers with magnitudes as large as approximately 10^{308} and as small as approximately 10^{-323} . Literals beyond these limits will be represented as accurately as possible. For example, if a literal represents a value that is too large for the Number data type to represent accurately, then the literal will evaluate to the special Number value Infinity.

A string literal is a quoted string of characters all on a single line, and, as in HTML, the quoting characters may be either a pair of single quotes or a pair of double quotes. Note that this differs from Java, where a character contained in single quotes belongs to

the `char` data type. It is much more efficient to store single characters using `char` than it is to use the Java `String` class. JavaScript, on the other hand, has only one data type for strings—named `String`—just as it has only one data type for numbers. This is another example of how JavaScript’s design favors simplicity over efficiency.

Like Java, escape codes can be embedded in string literals to represent various characters. For example, `\n` represents the line feed (newline) character, and `\"` (`\'`) can be used to represent a double (single) quote within a string literal that is enclosed in double (single) quotes. Of course, because the backslash character is special, it must be escaped within string literals; the escape code `\\` can be used for this purpose. Finally, any 16-bit Unicode character value may be represented by immediately following the `\u` escape code with four hexadecimal digits. For instance, `\u005c` represents the backslash character.

4.9 Functions

The structure of the high-low program of Figure 4.4 is not typical of many JavaScript programs designed to be run within browsers. Instead, such programs often are structured as a set of functions. A *function* in JavaScript is much like a method in Java: it consists of a function name, an argument list, and code that is executed when the function is called. However, unlike Java methods, a function does not need to be associated with an object.

To illustrate, consider Figure 4.8, which is the initial portion of a revision of the high-low program from Figure 4.4 (the remainder of the revised `HighLowWithFunction.js` program is identical to the corresponding portion of the original `HighLow.js`). The code beginning with the keyword `function` and ending with the next closing brace is a *function declaration*. Every JavaScript function declaration follows the same syntax: the keyword `function` followed by an identifier followed by a parenthesized list of zero or more comma-separated identifiers followed by a curly-brace bracketed sequence of statements. The first identifier is the *function name*, the list of identifiers is the function’s *formal parameter list*,

```
// HighLowWithFunction.js

var thinkingOf; // Number the computer has chosen (1 through 1000)
var guess;      // User's latest guess

// This function returns a random number between 1 and
// the argument value 'high'.
function oneTo(high) {
    return Math.ceil(Math.random()*high);
}

// Initialize the computer's number
thinkingOf = oneTo(1000);

...
```

FIGURE 4.8 Initial portion of JavaScript high-low game program that defines and uses a function named `oneTo()` for computing a random number between 1 and a specified value.

Client-Side Programming

and the statements represent the *function body*. In this example, the function name is `oneTo`, there is a single formal parameter named `high`, and the function body consists of a single return-value statement.

Unlike Java and C++, JavaScript does not require (or even allow) the data types of formal parameters or the data type of the return value of a function to be declared. (This shouldn't surprise us, since the data types of variables are also not declared in JavaScript.)

The code

```
oneTo(1000)
```

is a type of JavaScript expression known as a *function call*. Syntactically, the typical function call consists of an identifier followed by a parenthesized list of zero or more comma-separated *arguments*. In the example, the argument is `1000`.

When a scripting engine evaluates a function call, the effect is similar to that of a method call in Java. First, the engine evaluates each of the arguments, producing a value for each. In our example, `1000` is a `Number` literal and therefore evaluates to a `Number` value. If the function call had instead been written as `oneTo(999+1)`, the result of the function call would be the same, since the argument value would still be `1000`. Next, after evaluating the arguments, the resulting values are associated with the appropriate formal parameters. In our example, the value `1000` will be associated with the formal parameter `high`. Next, the body of the function is executed. Finally, the value of the function call expression is determined as follows. If a return-value statement is executed, then the value of the expression in the return-value statement becomes the value of the function call expression. On the other hand, if a return-void statement is executed or if the final statement in the body has been executed and is not a return statement, then the `Undefined` value becomes the value of the function call expression. In our example, a return-value statement will be executed, and the value of the expression in this statement is of type `Number`. So a `Number` value will be assigned to `thinkingOf`.

As in Java, code within a JavaScript function body can assign values to the function's formal parameters, and such assignments will not change the values of any variables in the function call's argument list, even if the variable and the parameter use the same identifier. For example, the following JavaScript program will produce two alert boxes, the first with the message `hi` and the second with the message `bye`:

```
// ArgChange.js

var message = "bye";

function change(message) {
    message = "hi";
    window.alert(message);
    return;
}

change(message);
window.alert(message);
```

As we'll learn later, JavaScript's handling of object and array arguments is also similar to Java's.

In Java, a compile error is generated if a method call's arguments do not match the number and data types of the formal parameters of the method. In JavaScript, the arguments and parameters do not have data types, so there is no need to match their types. What's more, a function call does not need to have the same number of arguments as there are formal parameters in the function declaration. If too few arguments are supplied, the formal parameters without arguments will be given the Undefined value. If too many arguments are supplied, the excess arguments are ignored.

If a `var` statement appears within a function body, then the variable declared is called a *local variable*, while variables that are declared outside of any function are called *global variables*. The relationship between global and local variables is very similar to the relationship between the instance variables of a Java object—those variables declared outside of any method of the class—and the variables declared within its methods. Global variables can be accessed from any part of a program, while local variables can only be accessed from the function that declares them. Also, global variables exist from the beginning of execution of a program until the program terminates, while a local variable exists only from the time the function declaring the variable is called until the function returns (see Section 4.10.5 for an exception to this rule). If a function is called multiple times, new copies of its local variables are created every time the function is called.

The following program demonstrates JavaScript's behavior if a local and a global variable share the same identifier:

```
// LocalScope.js
var j=6;    // global variable declaration and initialization
function test()
{
    var j;    // local variable declaration
    j=7;      // which variable(s) does this change?
    return;
}
test();
window.alert(j);
```

When the program is run, it displays an alert box containing 6, even though the `test()` function is called and assigns the value 7 to a variable named `j`. This assignment does not affect the value of the global variable by the same name because of JavaScript's *scope rules*, which dictate that if a variable reference is ambiguous (could refer to either a local or a global variable) then the scripting engine should associate the reference with the local variable. In such a situation, the global variable is said to be *shadowed* by the local variable with the same name.

To override JavaScript's default scope rules and access a shadowed global variable, you can prefix the name of the variable with `window`. Thus, if we changed the assignment statement within `test()` to

```
window.j = 7;
```

then the global rather than the local variable would be affected, and the output of the program would be 7 rather than 6. This works because global variables (as well as declared functions) are stored as properties of the `window` object. You'll want to keep this in mind if you run a debugger on a JavaScript program.

JavaScript supports recursive function calls. If a function calls itself recursively, each recursive execution will have its own local variables and formal parameter values. Thus, a recursive execution of a function does not have direct access to the parameters or local variables of other in-progress executions of the function. So, if you're familiar with the concept of a static local variable in C++ (a variable whose value is shared between calls to a recursive function or between successive nonrecursive calls to the same function), you should be aware that JavaScript has no direct support for this concept.

What if one function calls another function: does it matter which function is declared first in the program? No, it does not matter. This is because the scripting engine operates on a program in two *passes*, or phases. During the first pass, the scripting engine processes all `var` statements and function declarations. This creates global variables and function objects but doesn't actually execute any statements. In the second pass, the engine executes statements. So you can declare functions and variables in any order, because they will all be known to the scripting engine before it begins executing any statements that refer to the functions or variables.

Finally, JavaScript supplies some built-in functions that can be called just as you would call functions you have declared. In particular, the built-in functions `Boolean()`, `String()`, and `Number()` can be called to convert a value from any data type to a Boolean, String, or Number, respectively. Each function takes a single argument of any data type and returns a value of the specified type by applying the appropriate rules from Table 4.2, 4.3, or 4.4.

4.10 Objects

At this point, we've covered JavaScript's basic syntax, and we know a good deal about working with values belonging to the primitive (non-Object) data types. We'll now turn to JavaScript objects. The differences between JavaScript and Java are especially pronounced when it comes to objects, so if you've been skimming this chapter you may want to read this section more carefully.

4.10.1 Object Properties

An *object* in JavaScript is a set of *properties*, each of which consists of a unique *name* along with a *value* belonging to one of JavaScript's six data types. Properties in JavaScript are comparable to Java instance variables (i.e., nonstatic variables declared outside any method).

Like JavaScript variables, object properties themselves do not have data types; only the values assigned to properties have data types. For example, the following sequence of statements that successively assign Boolean, String, and Number values to a single property `prop` of an object `o` (assumed to be declared earlier) is syntactically valid in JavaScript, but would cause a compile error in Java or C++ regardless of how `o` was declared:

```
o.prop = true;  
o.prop = "true";  
o.prop = 1;
```

An even bigger difference with Java and C++ is that JavaScript programs do not define classes. There are some classlike features. For example, object constructors can be defined (as described later in this section) to create objects and automatically define properties for the objects created. Also, JavaScript uses a so-called *prototype* mechanism to provide a form of inheritance (this is an advanced topic that will not be covered in this chapter). However, unlike Java and C++, in which the class of an object defines its variables and methods, properties and methods can be added to or removed from a JavaScript object after it has been created.

For example, the following is valid in JavaScript:

```
var o1 = new Object();  
o1.testing = "This is a test";  
delete o1.testing;
```

We'll go into detail on these statements later in this subsection; for now, here's a brief description. The first line creates a variable named `o1` and initializes it with a value of type `Object` by calling on the built-in constructor `Object()` using a JavaScript `new` expression. The second line then adds a property named `testing` to the `o1` object and assigns a `String` value to this property. And the third line then deletes this property from `o1`.

Dynamic property creation is yet another example of JavaScript's flexibility, but also of its lack of certain safety features: if you misspell a property name in an assignment statement, you won't get an error message, but instead you'll create a new property. This is something to watch for when you're debugging JavaScript code.

Now we'll cover object creation and dynamic creation and removal of properties in somewhat more detail. First, as shown in the example, expressions beginning with the keyword `new` are used to create objects. Syntactically, a `new` expression begins with the `new` keyword followed by an identifier corresponding to an object constructor followed by a parenthesized list of zero or more arguments. We'll learn how to create user-defined constructors later in this chapter.

A `new` expression causes a new empty object to be created and then calls the specified constructor, supplying it with this new object plus the specified argument values. The constructor can then perform initialization on the object, which might involve creating and initializing properties, adding methods to the object, and adding the object to an inheritance hierarchy from which it can inherit additional properties and methods. In the case of the `Object()` constructor, no properties or methods are added directly to the new object by the constructor, but the object is modified so that it inherits several generic methods including default `toString()` and `valueOf()` methods used when converting the object to `String` and `Number` values, respectively. The values produced by these default methods are not particularly useful, but they at least prevent a run-time error in case an attempt is made to apply data type conversion to the object.

We next turn to property creation. As we saw, when a JavaScript statement attempts to assign a value to an object property, and the property does not exist in the object, then a property with the given name is created in the object and assigned the specified value. This happens even if the object has inherited a property with the same name, since an inherited property or method actually resides in a different object.

Finally, as shown, the keyword `delete` can be used to remove a property from an object. Syntactically, a `delete` expression begins with the `delete` unary operator followed by a reference to an object property, such as `o.testing`. You should not attempt to delete a nonexistent property (technically, this should be legal, but some browsers will throw an exception).

JavaScript provides a handy shortcut called an *object initializer* for creating an empty object (as if by a call to `new Object()`), creating properties on this object, and assigning values to these properties. For example, the statement

```
var o2 = { p1:5+9, p2:null, testing:"This is a test" };
```

creates an object with three properties `p1`, `p2`, and `testing` and assigns these properties the values `14`, `null`, and `this is a test`, respectively. A reference to this object is then assigned to the variable `o2`.

4.10.2 Enumerating Properties

If properties can be dynamically created and destroyed, how can your program know which properties an object has at any given time? JavaScript provides a special `for-in` statement that can be used to iterate through all of the property names of an object. The following JavaScript code illustrates the use of this statement:

```
var hash = new Object();
hash.kim = "85";
hash.sam = "92";
hash.lynn = "78";
for (var aName in hash) {
    window.alert(aName+" is a property of hash.");
}
```

Executing this program will produce three alert boxes, each with one of the names `kim`, `sam`, or `lynn`. However, the order in which they appear is implementation dependent.

The syntax of a `for-in` statement is as shown: It begins with the reserved word `for` followed by a parenthesized portion followed by a statement (often a brace-enclosed block statement). The parenthesized portion begins with a variable identifier, optionally preceded by `var` if the variable was not previously declared. Then comes the reserved word `in` followed by an expression that evaluates to an object.

4.10.3 Array Notation

The code in the preceding example prints the property names. What if we want to print the values of these properties? The notation `hash.aname` represents the property named `aname`, which is not what we want. What we want is a notation that means “Evaluate the expression

`aName` and convert it to a `String` if necessary. Then retrieve the property that has this string value as its name from the `hash` object.”

As you might guess, JavaScript provides just such a notation: `hash[aName]` has exactly this meaning. So, for example, if we change the `window.alert()` call in the code to

```
window.alert(aName + " scored " + hash[aName]);
```

then the alert boxes output by the program will contain both the name and the value for each of the three properties in `hash`.

In short, JavaScript provides two different notations for accessing object properties. The first notation is the familiar dot notation, for example `hash.kim`. The second notation uses an array reference syntax in which the index is viewed as a `String` value, for example `hash["kim"]`. As we have seen, the array notation has the advantage that any expression, not just a string literal, can be used to represent the property name. So the array notation is more general, and in fact scripting engines view dot notation such as `hash.kim` as shorthand for the equivalent array notation `hash["kim"]`.

Therefore, an object in JavaScript can also be viewed as a sort of array in which the elements are indexed by strings. Such an array is sometimes called an *associative array*. This is a powerful form of data structure, similar to Java’s `java.util.Hashtable` class but built directly into the JavaScript language rather than provided by an API. In addition to the associative array feature of all JavaScript objects, JavaScript objects constructed using the `Array()` constructor (covered in Section 4.11) also have additional array-specific properties and methods.

4.10.4 Object References

Recall that if you execute the Java code

```
StringBuffer s1 = new StringBuffer("Hello");  
StringBuffer s2 = s1;
```

then a single `StringBuffer` is created and both `s1` and `s2` will be references to it. This is because what is stored in a Java variable that represents an object is a reference (pointer) to the object and not the object itself. Thus, the second statement in the code copies the reference from `s1` to `s2` rather than making a copy of the entire object. Therefore, if the code is followed by

```
s2.append(" world!");  
System.out.println(s1);
```

then even though it might appear that `s2` is modified and `s1` is not, the output will be `Hello world!`.

Client-Side Programming

In the same way, JavaScript values of type `Object` are also references to objects, not actual objects themselves. So the following JavaScript code will also output `Hello world!`:

```
// ObjRef.js
var o1 = new Object();
o1.data = "Hello";
var o2 = o1;
o2.data += " world!";
window.alert(o1.data);
```

Similarly, when an `Object` value is used as an argument to a function or method, the object reference is passed and not a copy of the object itself. To illustrate this, consider the JavaScript program of Figure 4.9. The program creates two objects `o1` and `o2` and passes them as arguments to the function `objArgs()`. Thus, immediately after the function is called, `param1` is a copy of the object reference contained in `o1`, and `param2` a copy of `o2` (part (a) of Figure 4.10). The first statement of the function changes the `data` property of the object referenced by `param1` and `o1` (Figure 4.10(b)). The function then changes `param2` to be a copy of the object reference contained in `param1`. Note that this has no impact on the variable `o2` or on the object referenced by this variable (Figure 4.10(c)). The output of the program will therefore be the two alert boxes shown in Figure 4.11.

```
// ObjArg.js

function objArgs(param1, param2) {
    // Change the data in param1 and its argument
    param1.data = "changed";
    // Change the object referenced by param2, but not its argument
    param2 = param1;

    window.alert("param1 is " + param1.data + "\n" +
                 "param2 is " + param2.data);
    return;
}

// Create two different objects with identical data
var o1 = new Object();
o1.data = "original";
var o2 = new Object();
o2.data = "original";

// Call the function on these objects and display the results
objArgs(o1, o2);
window.alert("o1 is " + o1.data + "\n" +
             "o2 is " + o2.data);
```

FIGURE 4.9 JavaScript program illustrating the use of objects as arguments.

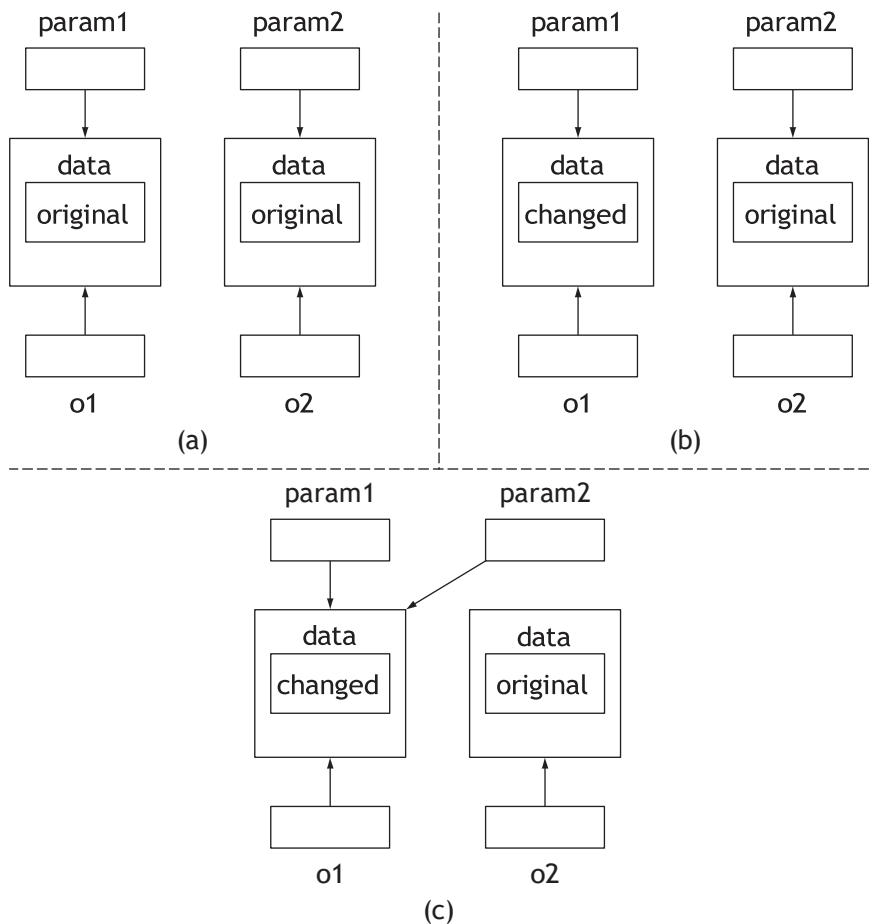


FIGURE 4.10 State of variables and parameters of the `ObjArg.js` program (a) immediately before the first statement of the function `objArgs()` is executed; (b) immediately after this statement is executed; and (c) immediately after the second statement of the function is executed.

4.10.5 Methods

If you read the description of the `typeof` operator (Table 4.1) carefully, you noticed that a value of type `Object` can represent a function. In fact, internally, JavaScript represents every function as an `Object` value. That is, when JavaScript processes a function declaration such as the one for `objArgs()`, it creates a specialized object that represents the function. A variable that has the same name as the function is also created, and a reference to the function object is assigned to this variable. Thus, when the program of Figure 4.9 is executed, it automatically creates a variable named `objArgs` that is a reference to an `Object` value representing the function `objArgs()`.

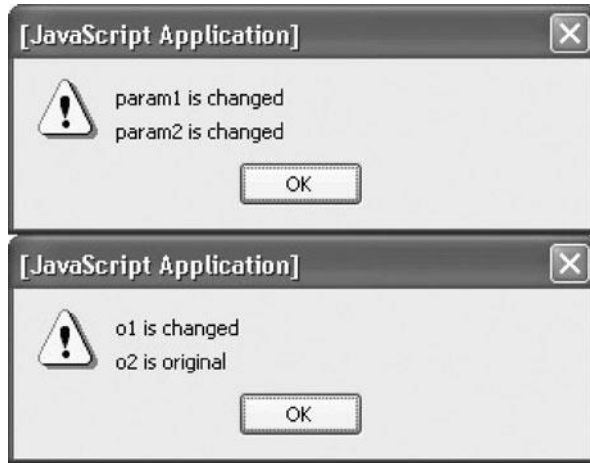


FIGURE 4.11 The two output alert windows produced by running `ObjArg.js`.

A *method* in JavaScript is simply a function that has been assigned as the value of a property of an object. For example, consider the JavaScript code of Figure 4.12, which creates a `node` object that could be suitable for use in a binary search tree data structure (see Section 4.10.7 for more on binary search trees). The code begins by declaring a function named `leaf()` that returns `true` exactly when a node represents a leaf (i.e., when both its `left` and `right` properties have the value `null`). As just noted, this function declaration also creates a variable named `leaf` that contains a reference to the object representing the function. The program subsequently contains the statement

```
node.isLeaf = leaf;
```

This creates a property named `isLeaf` and assigns to it a reference to the `leaf` function object. In other words, this creates a method on the `node` object. Although the method name can be the same as the function name, this is not necessary. The `isLeaf()` method is called later in the program with expressions such as

```
node1.isLeaf()
```

If you run the program, you'll see that the first node is not a leaf and the second one is a leaf.

As illustrated in the body of `leaf()`, the JavaScript keyword `this` is used to allow a method to access other properties of the object to which the method is assigned. That is, if `this` is used in an expression in the body of a function that is a method on some object, then when the method is called, `this` will evaluate to a reference to the object. So, when the `isLeaf()` method is called on `node 1` causing `leaf()` to be called, the expression `this` within `leaf()` evaluates to a reference to `node 1`.

```

// NodewithMethod.js

// leaf() returns true iff this node is a leaf node.
// It is designed to be called as a method, not directly
// as a function.
function leaf() {
    return this.left == null && this.right == null;
}

// makeBTNode(value) creates a binary tree node and
// initializes its value to the given argument.
// It also adds an isLeaf() method to the node.
function makeBTNode(value) {
    var node = new Object();
    node.left = node.right = null;
    node.value = value;
    node.isLeaf = leaf;
    return node;
}

// Create and initialize two node objects, making the second
// a child of the first.
var node1 = makeBTNode(3);
var node2 = makeBTNode(7);
node1.right = node2;

// Output the value of isLeaf() on each node
window.alert("node1 is a leaf: " + node1.isLeaf());
window.alert("node2 is a leaf: " + node2.isLeaf());

```

FIGURE 4.12 Program that creates a node object containing a `clear()` method.

Notice that the function `leaf()` is declared as a function but is intended to be used only as a method, not a function. To make this intention clear (which could be helpful to someone else maintaining this program, for example), it would be better if we could avoid declaring the function at all and instead simply create a method directly. In fact, we can do this by removing the declaration of `leaf()` and replacing the assignment to `node.isLeaf` as follows:

```

// NodewithMethod2.js
// Remove leaf() declaration found in NodewithMethod.js
...
node.isLeaf =
    function leaf() {
        return this.left == null && this.right == null;
    };

```

When a function keyword appears where an expression is expected, such as on the right-hand side of an assignment statement, it marks the beginning of a *function expression*. Such an expression evaluates to an Object value representing a function, just as a function declaration does. However, a key difference is that the scripting engine does not create

a variable for the object created by a function expression. Therefore, if we create the `isLeaf()` method using a function expression, then it is no longer possible to call `leaf()` directly.

I should explain one syntactic aspect of this code: Notice that there is a semicolon following the final closing brace of the `leaf()` function expression. This semicolon marks the end of the statement that assigns a value to the `isLeaf` property. While JavaScript does not always require this (since the syntax rules often allow semicolons to be omitted), it's good practice to include it.

One final note: If a function declares a local variable and also evaluates a function expression, and if the function expression contains a reference to the local variable, then the local variable will continue to exist after the function declaring the variable returns.

4.10.6 Constructors

The function `makeBTNode()` in the previous example acts as a kind of constructor for objects representing nodes in a binary search tree. While this code is fairly simple, it can be made even simpler by using JavaScript's constructor mechanism.

Figure 4.13 illustrates the use of a JavaScript constructor. It will help you to read this code if you understand that every function can be called not only as a function and as

```
// BTNode.js

// BTNode(value) is a constructor for a binary tree node.
// It initializes its value to the given argument.
// It also adds an isLeaf() method to the node.
function BTNode(value) {
    // Notice that we no longer need to create an Object
    // and that we use "this" to reference the object
    // initialized.
    this.left = this.right = null;
    this.value = value;
    this.isLeaf =
        function leaf() {
            return this.left == null && this.right == null;
        };
    // Notice that we no longer return a value.
}
// Create and initialize two node objects, making the second
// a child of the first.
// Notice the use of "new" to call a function as a constructor.
var node1 = new BTNode(3);
var node2 = new BTNode(7);
node1.right = node2;

// Output the value of isLeaf() on each node
window.alert("node1 is a leaf: " + node1.isLeaf());
window.alert("node2 is a leaf: " + node2.isLeaf());
```

FIGURE 4.13 Program that defines and uses an object constructor.

a method, but also as a constructor. That is, every JavaScript function you declare is also automatically a constructor. Therefore, the `BTNode()` function declared at the beginning of Figure 4.13 can be used as a constructor, even though its declaration is no different than any other function declaration.

You tell the scripting engine that you want to call a function as a constructor by prefixing the call with the `new` keyword. Thus, in the statements initializing `node1` and `node2`, we are calling `BTNode()` as a constructor rather than as a standard function. When a function is called as a constructor, the scripting engine automatically creates an empty object and associates the `this` keyword with this object within the body of the function. This is why we do not need to explicitly create an object in the `BTNode()` function as we did in the earlier `makeBTNode()` function. This is also why we use the keyword `this` in the assignment statements in the body of `BTNode()`.

Finally, although the `BTNode()` function does not return a value, each `new` expression used to call this function as a constructor will return as its value the object that was automatically created by the scripting engine and initialized by the code in the body of `BTNode()`. So, as before, the output of this program will show that the first node is not a leaf and the second is.

Thus, although JavaScript does not have a class concept exactly like that found in Java and C++, we see that the constructor mechanism can be used to provide somewhat similar functionality. In fact, when we construct an object in JavaScript using `new`, the constructed object is known as an *instance* of the function (object) used as the constructor. In the preceding example, we could call `node1` and `node2` instances of `BTNode`. The JavaScript `instanceof` operator can be used to test whether one object is an instance of another object. For example, adding the code

```
// Test that node1 is an instance of BTNode
window.alert("node1 is instance of BTNode: " +
    (node1 instanceof BTNode));
```

to the end of the `BTNode.js` file produces the output `node1 is instance of BTNode: true`.

4.10.7 Example: Binary Tree

We'll end this section on JavaScript objects by writing code for creating and using a simple binary search tree. Recall that in a binary tree, every node has a left and a right child, although either or both children may be empty (null). In a binary search tree, each node *N* has the property that all values (if any) stored in nodes in the left subtree of *N* are smaller than *N*'s value, and all values stored in its right subtree are larger than *N*'s value. Therefore, we can search to see whether or not a value is stored in such a tree by first comparing the search value with the root node's value. If the values match, we're done. Otherwise, if the search value is smaller than the root value, we continue our search in the left subtree, because we know that the search value must be there if it is in the tree at all. Similarly, we need only search the right subtree if the search value is larger than the root value.

Figures 4.14 and 4.15 show JavaScript code for a constructor function `searchTree()` that creates a binary search tree object having methods `insert()` and `search()`. The

Client-Side Programming

```
// SearchTree.js (part 1)

// Constructor for BTreeNode objects
function BTreeNode(value) {
  ...
}

// Constructor for SearchTree objects
function SearchTree() {

  /*** Variables ***/
  this.root = null;

  /*** Methods ***/

  // locate(value) returns a reference to the node
  // containing the value if the given value is in the search tree,
  // returns null if the search tree is empty,
  // and otherwise returns a reference to the node that
  // should be this value's parent.
  // This method is only intended to be used by other
  // methods, not called directly.
  this.locate =
    function locate(value) {
      var curr = this.root; // Node currently being visited
      var parent = null;    // Parent of current node
      // Search for value, remembering parent nodes
      while (curr != null && curr.value != value) {
        parent = curr;
        if (value < parent.value) {
          curr = parent.left;
        }
        else {
          curr = parent.right;
        }
      }

      // If curr is null, we did not locate the value.
      // Return parent (which is null if tree is empty) instead.
      if (curr == null) {
        curr = parent;
      }
      return curr;
    };
};
```

FIGURE 4.14 Initial portion of program for creating a binary search tree.

`insert()` method first searches for the value to be inserted and, if the value is not found, creates a `BTreeNode` (using the constructor of Figure 4.13). It then inserts this node into an appropriate location so that the tree retains the binary search property described in the preceding paragraph. Both methods rely on another method named `locate()` that performs the actual search for a value in the tree and returns a value that either indicates that the search

```

// SearchTree.js (part 2)

// insert(value) adds the given value to the search
// tree if it is not already present.
this.insert =
  function insert(value) {

    // Search for location of value or its parent
    var location = this.locate(value);

    // If value is not present (location is non-null
    // and location.value is not same as given value),
    // then create a node containing the value
    // and insert it at the appropriate location.
    var node = new BTNode(value);
    if (location == null) { // Empty tree
      this.root = node;
    }
    else if (location.value != value) { // location is parent
      if (value < location.value) {
        location.left = node;
      }
      else {
        location.right = node;
      }
    }
  }
};

// search(value) returns true iff the given value is
// present in the tree
this.search =
  function search(value) {

    // Search for location of value or its parent
    var location = this.locate(value);

    // Found value iff location is not null and value
    // of location is the given value
    return location != null && location.value == value;
  }
};

```

FIGURE 4.15 Remainder of constructor for binary search tree program.

succeeded or provides information that can be used to insert the value into the appropriate location within the tree. Figure 4.16 then shows code that constructs a tree, inserts values into the tree, and searches the tree. When run, this code will indicate that the value 12 is contained in the tree but that 6 is not.

This program illustrates several points. First, notice that a method can declare local variables (such as `curr` in `locate()`) just like any other function, and that these variables are referenced within a method just as they would be within a normal function.

Client-Side Programming

```
// SearchTree.js (part 3)

// Create a search tree and insert values
var tree = new SearchTree();
tree.insert(7);
tree.insert(3);
tree.insert(12);
tree.insert(12); // Try inserting a second time

// Search the tree
window.alert("Search for 12 produces " + tree.search(12));
window.alert("Search for 6 produces " + tree.search(6));
```

FIGURE 4.16 Code for using binary search tree constructor and calling methods on constructed tree object.

Second, note that if one method (such as `insert()`) wants to call another method (such as `locate()`) of the same object, the call must be prefixed with `this`. Otherwise, the scripting engine would interpret a call without the leading `this`—such as `locate(value)`—as a call to a regular function and not to a method. Third, notice that although `locate()` is intended to be called only by other methods within a `searchTree` object, there is no `private` declaration in JavaScript to enforce such an intention. Since most JavaScript programs are small, this simplification compared with Java and C++ is a reasonable trade-off.

Although the test program in Figure 4.16 uses integer values for nodes, notice that `searchTree` and `BTNode` objects can accommodate values of any type. In fact, values of different types can be stored in a single `searchTree`, because JavaScript’s comparison operators (`!=`, `<`, etc.) can be used to compare arbitrary data types, as explained in Section 4.7. However, note that because the code uses the nonstrict equality operators `!=` and `==`, values belonging to two different data types might map to a single node. For example, if the statement

```
window.alert('Search for "3" produces ' + tree.search("3"));
```

is added to the end of the `searchTree.js` program, this search will evaluate to `true`, even though no node in the tree contains the String value 3. We could avoid this effect by using the strict equality operators `!==` and `===` in place of the nonstrict operators in the comparisons involving the `value` property of `BTNode`’s.

4.11 Arrays

We learned earlier that all object properties can be accessed using an arraylike notation, and that objects can therefore be viewed as associative arrays. In addition, JavaScript supplies a native function object named `Array` that can be used to construct objects that have special array characteristics and that inherit a number of array-oriented methods. In this section, we’ll learn more about creating and using `Array` instances, which we’ll refer to simply as *arrays*.

4.11.1 Creating an Array

One way to create an array is to use the `Array` constructor directly in a call with no arguments:

```
var ary1 = new Array();
```

Alternatively, an array can be constructed and initialized with values by supplying two or more arguments to the `Array` constructor:

```
var ary2 = new Array(4, true, "ok");
```

After this statement is executed, `ary2[0]` will have the `Number` value 4, `ary2[1]` the `Boolean` value `true`, and `ary2[2]` the `String` value `ok`. Keep in mind that array notation is just a way of specifying the name of a property in JavaScript. So an expression such as `ary2[0]` evaluates to the value of the property with the name 0 (a `String` consisting of a single numeric character) belonging to the object referenced by the variable `ary2`. On the other hand, attempting to execute the expression `ary2.0` would generate a syntax error in JavaScript. This is because an identifier must be used for the property name in dot notation, and the syntax rules for identifiers require that they must not begin with a numeric character. There is, however, no such restriction on the actual names that can be used for properties: any string can be used.

I will refer to properties of an array object that have names that are string representations of natural numbers as the *elements* of the array. So `ary2` in the second example has three elements immediately after the `Array` constructor is executed. An array object can also have other nonelement properties. In fact, every array object is automatically given a special property named `length`. When an array is constructed with a no-argument constructor (as in the case of `ary1` in the first example), the value of `length` will be 0. When constructed with two or more arguments, `length` is set to the number of arguments. Thus, after the statement creating the variable `ary2`, `ary2.length` will have the value 3. We'll have more to say about this property in the next section.

Another way to create and initialize an array that produces identical results to those of the previous example is

```
var ary3 = [4, true, "ok"];
```

An expression such as the one on the right side of the assignment in this example is called an *array initializer*. This is essentially a shorthand that implicitly calls the `Array` constructor for us.

The elements of a JavaScript array can be any JavaScript values, including other instances of `Array`. This feature can be used to create multidimensional arrays. For example, one way to create a two-dimensional array is

```
// array2d.js
var ttt = [ [ "x", "0", "0" ],
            [ "0", "x", "0" ],
            [ "0", "x", "x" ] ];
```

In this example, the outer one-dimensional array (named `ttt`) contains three elements, each itself a one-dimensional array. The notation `ttt[1]` will evaluate to a reference to the second of these internal arrays (the `o x o` array). Since this is an array object itself, the notation `ttt[1][2]` will evaluate to the value of the third element of this array, `o`. Higher-dimensional arrays can be created and accessed similarly.

4.11.2 Dynamically Changing Array Length

Just as we can dynamically add properties to any JavaScript object, we can add properties to an array. In particular, in Section 4.11.1, we could follow the code creating `ary2` (which had three elements) with the statement

```
ary2[3] = -12.6;
```

This will create a new property named (as a String) `3` with the value `-12.6`. In addition, the `length` property of `ary2` will be changed to `4`. Thus, unlike arrays in Java, the effective size of JavaScript arrays can change dynamically. In this way, JavaScript arrays are more like instances of the `java.util.Vector` class than they are like Java arrays.

Elements can also be removed from a JavaScript array by decreasing the value of the `length` property. For example, if we next executed the statement

```
ary2.length = 2;
```

then the properties `2` and `3` would automatically be removed from `ary2`.

While we're on the topic of the `length` property, let me mention a special case that might cause you trouble if you're not aware of it. If the `Array` constructor is called with a single `Number` argument, then this argument value is assigned to the `length` property but no array elements are actually created. As an example, the expression

```
new Array(200)
```

creates an array having `length` `200` but does not actually create or initialize the properties with names `0` through `199`. Similarly, if you increase the `length` value of an array object, this does not automatically add any elements (properties with numeric names) to the array. Thus, your code should not assume that `length` always represents the actual number of elements in an array.

4.11.3 Array Methods

Every array object automatically inherits a number of useful methods. Many of these methods are described briefly in Table 4.7, and some additional details are given in this subsection. Full algorithmic details of every method are provided in Section 15.4.4 of the ECMAScript specification [ECMA-262].

The argument to the `sort()` method should be an object representing a function. This function in turn should take two arguments representing array elements and return a `Number` value. A negative return value indicates that the element corresponding to the first argument should come before the element corresponding to the second in the sorted array,

TABLE 4.7 Methods Inherited by Array Objects. Unless Otherwise Specified, Methods Return a Reference to the Array on Which They are Called.

Method	Description
toString()	Return a String value representing this array as a comma-separated list.
sort(Object)	Modify this array by sorting it, treating the Object argument as a function that specifies sort order (see text).
splice(Number, 0, any type)	Modify this array by adding the third argument as an element at the index given by the first argument, shifting elements up one index to make room for the new element.
splice(Number, Number)	Modify this array by removing a number of elements specified by the second argument (a positive integer), starting with the index specified by the first element, and shifting elements down to take the place of those elements removed. Returns an array of the elements removed.
push(any type)	Modify this array by appending an element having the given argument value. Returns length value for modified array.
pop()	Modify this array by removing its last element (the element at index length-1). Returns the value of the element removed.
shift()	Modify this array by removing its first element (the element at index 0) and shifting all remaining elements down one index. Returns the value of the element removed.

a positive value indicates that the second should come before the first, and 0 indicates that the elements can be in either order (for sorting purposes, they are equivalent). For example, in the program of Figure 4.17, the return value will be positive if the first argument is greater than the second and negative if the first is less than the second. This will result in an array of Number values being sorted into ascending order, as shown. Also notice that the argument to sort() can be a function expression rather than the name of a function declared elsewhere.

```
// ArrayMethods.js
var numArray = [1,3,8,4,9,7,6,2,5];

// Sort in ascending order
numArray.sort(
    function compare (first, second) {
        return first - second;
    }
);
// numArray.toString(): 1,2,3,4,5,6,7,8,9

numArray.splice(2, 0, 2.5);
// numArray.toString(): 1,2,2.5,3,4,5,6,7,8,9

// output of following: 5,6,7
window.alert(numArray.splice(5,3).toString());
// numArray.toString(): 1,2,2.5,3,4,8,9
window.alert(numArray.toString());
```

FIGURE 4.17 Program illustrating use of several of the methods inherited by array objects.

Figure 4.17 also illustrates two versions of the `splice()` method, which is used both for inserting an element into an array (and automatically making room for the new element by shifting other elements) and for removing one or more elements from an array (and automatically shifting other elements to take the place of those removed). Notice that the version of `splice()` that removes elements also returns an array containing the elements removed.

The `push()`, `pop()`, and `shift()` methods make it easy to implement basic stack and queue data structures using arrays (if you're not familiar with these data structures, you may want to either read about them in any textbook on data structures or skip the remainder of this section). In the case of a stack, the `length` property acts as the stack pointer, with an empty stack indicated by a 0 value. The following code illustrates usage of these methods to implement a stack:

```
// stack.js
var stack = new Array();

stack.push('H');
stack.push('i');
stack.push('!');

var c3 = stack.pop(); // pops '!'
var c2 = stack.pop(); // pops 'i'
var c1 = stack.pop(); // pops 'H'
window.alert(c1 + c2 + c3); // displays "Hi!"
```

You can similarly implement a basic queue structure using `push()` to add elements to the end of the queue and `shift()` to remove elements from the front of the queue. In this case, `length` will point to the back of the queue, and the front of the queue will be at index 0. Other array methods can be used for additional functionality. For example, `splice()` could be used to insert an element at a location other than the end of the queue, which might be useful as part of an implementation of priority queues.

4.12 Built-in Objects

In this section, we will briefly overview several of JavaScript's built-in objects. Chapter 15 of the ECMAScript reference [ECMA-262] describes all built-in objects and should be consulted for details.

4.12.1 The Global Object (`window`)

In every JavaScript host environment, some object is designated as the *global object*. In common browsers, this object is named `window`. It is called the global object because, as mentioned earlier, all global variables declared by your program are actually stored as properties of this object. What's more, other built-in objects, such as `object`, `Array`, and other objects described later, are also stored as properties of the global object. For example, the `object` object is stored in `window.object`.

Furthermore, all host objects are stored (directly or indirectly) as properties of the global object. I've emphasized this fact in the JavaScript code in this chapter by prefixing `window.` before the calls to the `alert()` and `prompt()` host methods. Usually, though, this isn't necessary, because if your code calls a function or refers to a variable and JavaScript can't locate the function or variable among your function declarations and local variables (and formal parameters, if the code is in a function body), then it will look for a property by the given name in the `window` object.

In addition to properties representing built-in and host objects, the global object contains several other useful properties. For instance, the global object property `Infinity` contains the `Number` value representing Infinity, which is sometimes useful (as an initial value in code for finding the minimum value in an array, for example). See the ECMAScript specification [ECMA-262] for additional global object properties.

4.12.2 String, Number, and Boolean

We've previously learned that JavaScript has built-in functions `String()`, `Number()`, and `Boolean()` that can be used to convert values to each of these data types. If any of these functions is called as a constructor, it will not only convert its argument value to the appropriate type if necessary, but will also wrap the converted value in an object that is an instance of the constructor object. For example,

```
// wrappedNumber.js
var wrappedNumber = new Number(5.625);
```

creates an object that is an instance of `Number`. This object will inherit certain methods associated with the `Number` object. One of these methods will be a version of `valueOf()` that returns the wrapped value (a value of type `Number` in the case of an instance of `Number`). So, for example, executing

```
// wrappedNumber.js (part 2)
window.alert(typeof wrappedNumber.valueOf());
```

will indicate that `wrappedNumber.valueOf()` is of type `Number`.

In addition to `valueOf()`, `Number` instances inherit several methods that can be used to format a numeric value for output purposes. For example,

```
// wrappedNumber.js (part 3)
window.alert(wrappedNumber.toFixed(2));
```

will output 5.63, which is the value of the original number rounded to two decimal places. The `toExponential(Number)` method is similar:

```
// wrappedNumber.js (part 4)
window.alert(wrappedNumber.toExponential(2));
```

Client-Side Programming

TABLE 4.8 Some of the Methods Inherited by `String` Instances

Method	Description
<code>charAt(Number)</code>	Return string consisting of single character at position (0-based) <code>Number</code> within this string.
<code>concat(String)</code>	Return concatenation of this string to <code>String</code> argument.
<code>indexOf(String, Number)</code>	Return location of leftmost occurrence of <code>String</code> within this string at or after character <code>Number</code> , or <code>-1</code> if no occurrence exists.
<code>replace(String, String)</code>	Return string obtained by replacing first occurrence of first <code>String</code> in this string with second <code>String</code> .
<code>slice(Number, Number)</code>	Return substring of this string starting at location given by first <code>Number</code> and ending one character before location given by second <code>Number</code> .
<code>toLowerCase()</code>	Return this string with each character having a Unicode Standard lowercase equivalent replaced by that character.
<code>toUpperCase()</code>	Return this string with each character having a Unicode Standard uppercase equivalent replaced by that character.

will output `5.63e+0`. Finally,

```
// wrappedNumber.js (part 5)
window.alert(wrappedNumber.toString(2));
```

outputs `101.101`, which is `5.625` represented in binary (base 2). This method can be used to output a number in any standard number base, including base 8 (octal) and base 16 (hexadecimal).

The `Number` object itself also provides properties `Number.MIN_VALUE` and `Number.MAX_VALUE`, which contain values that are the smallest and largest (in absolute value) numbers that can be represented in JavaScript.

Every `string` instance has a property named `length`, which is a `Number` value representing the number of characters in the internal string value. In addition, instances of `string` inherit a number of methods, including those shown in Table 4.8. Note that none of these methods change the value of the `string` instance on which they are called. Instead, they return a value that uses the instance's underlying `String` value as an input.

Finally, JavaScript's automatic type conversion allows you to apply `string` and `Number` inherited methods directly to primitive values of type `String` and `Number`, respectively. For example, the following is syntactically legal JavaScript that will output `str`:

```
// slicestring.js
var primitiveString = "a String value";
window.alert(primitiveString.slice(2,5));
```

When the scripting engine encounters this code, it automatically wraps the `String` value contained in `primitiveString` within a `string` object instance, since the `String` value is being used where an `Object` value is expected (preceding a dot). Thus, the code given is a shorthand for

```
// slicestring.js (part 2)
window.alert((new String(primitiveString)).slice(2,5));
```

In fact, you can even use a string literal as the “object” on which a string method is called:

```
// slicestring.js (part 3)
window.alert("a String value".slice(2,5));
```

4.12.3 Date

The expression

```
new Date()
```

returns an instance of the built-in `date` object representing the current date and time on the host machine (which, when JavaScript is run within a browser, is the machine running the browser). `date` instances can be used for a variety of purposes. One common use is to generate a string representing the current date and time in the user’s time zone; this string can then be displayed in the browser (using techniques discussed in Chapter 5) to make a web page appear fresh. The inherited `toLocaleString()` method, when called on a `date` instance, returns a date/time string formatted according to the conventions used in the user’s location (as determined by browser settings). Alternatively, `toLocaleDateString()` and `toLocaleTimeString()` can be used to obtain only the date or time, respectively. Other methods allow you to extract the year, month, day within the month or week, hour, and so on. See Section 15.9.5 of the ECMAScript reference [ECMA-262] for a complete list of inherited methods.

Another use of `date` instances is to determine how much time has elapsed between two events. For example, consider the following JavaScript code:

```
// ElapsedTime.js

var startTime = new Date();

// Perform some processing
...

var endTime = new Date();
window.alert("Processing required " +
             (endTime - startTime)/1000 +
             " seconds.");
```

Recall that when JavaScript attempts to perform arithmetic involving Object values (such as `date` instances), it first casts the objects to Number values by calling `valueOf()` on each object. The `valueOf()` method inherited by a `date` instance returns an integer representing the number of milliseconds that have elapsed between the date and time represented by the instance and a certain fixed date and time (midnight universal (UTC) time on January 1, 1970). So taking the difference of two `date` instances gives the number of milliseconds

that have elapsed from the date and time represented by the first instance to the date and time of the second. Dividing this difference by 1000 gives the elapsed time in seconds.

Similarly, if `date1` and `date2` are `Date` instances, then the following code will execute the block statement controlled by the `if` statement only if the date and time represented by `date1` is prior to that represented by `date2`:

```
if (date1 < date2) { ... }
```

You should understand why this code works in view of the preceding discussion. It's worth noting that to compare dates in Java requires a method call. Thus, this code once again illustrates how intuitive JavaScript code can be due to its use of automatic type conversion.

4.12.4 Math

The `Math` object is like the `global` object in that you do not construct instances from it, but instead you call methods directly on it. For instance, to perform a square root operation, you can use an expression such as

```
Math.sqrt(15.3)
```

All of the methods of this object are listed in Table 4.9. In addition to these methods, `Math` has several properties with values of type `Number`, including `E` and `PI`, which represent

TABLE 4.9 Methods of the `Math` Built-in Object

Method	Return Value
<code>abs(Number)</code>	Absolute value of <code>Number</code>
<code>acos(Number)</code>	Arc cosine of <code>Number</code> (treated as radians)
<code>asin(Number)</code>	Arcsine of <code>Number</code>
<code>atan(Number)</code>	Arctangent of <code>Number</code> (range <code>-Math.PI/2</code> to <code>Math.PI/2</code>)
<code>atan2(Number, Number)</code>	Arctangent of first <code>Number</code> divided by second (range <code>-Math.PI</code> to <code>Math.PI</code>)
<code>ceil(Number)</code>	Smallest integer no greater than <code>Number</code>
<code>cos(Number)</code>	Cosine of <code>Number</code> (in radians)
<code>exp(Number)</code>	<code>Math.E</code> raised to power <code>Number</code>
<code>floor(Number)</code>	Largest integer no less than <code>Number</code>
<code>log(Number)</code>	Natural logarithm of <code>Number</code>
<code>max(Number, Number, ...)</code>	Maximum of given values
<code>min(Number, Number, ...)</code>	Minimum of given values
<code>pow(Number, Number)</code>	First <code>Number</code> raised to power of second <code>Number</code>
<code>random()</code>	Pseudo-random floating point number in range 0 to 1
<code>round(Number)</code>	Nearest integer value to <code>Number</code>
<code>sin(Number)</code>	Sine of <code>Number</code>
<code>sqrt(Number)</code>	Square root of <code>Number</code>
<code>tan(Number)</code>	Tangent of <code>Number</code>

the base of the natural logarithm and the ratio of the circumference to the diameter of a circle, respectively.

4.12.5 RegExp

JavaScript provides regular expression capabilities through instances of the built-in `RegExp` object. A *regular expression* is a certain way of representing a set of strings. Regular expressions are frequently used to test that a string entered in an HTML form has a certain format, or, in the terminology of regular expressions, belongs to the set of strings that have the correct format.

As a simple example, the set of strings that consist of exactly three digits—which might represent, for example, the set of valid area codes in a phone number—can be represented by the JavaScript regular expression

```
\d\d\d
```

In a JavaScript regular expression, the string `\d` stands for the set of digit characters 0 through 9. Concatenating this string with itself three times as shown represents the set of all possible strings of three consecutive digits.

We could use this regular expression as follows to test whether or not a `String` value contained in a variable named `areaCode` consists of exactly three digits:

```
var acTest = new RegExp("^\\d\\d\\d$");
if (!acTest.test(areaCode)) {
    window.alert(areaCode+" is not a valid area code.");
}
```

(We'll soon see why this regular expression doesn't look exactly like the preceding one.) The `test()` method of a `RegExp` instance returns `true` if its `String` argument is a member of the set of strings represented by the instance's regular expression, and returns `false` otherwise. In this example, we want to output a message if `areaCode` is not in the set of valid area codes, so we complement the value returned by `test()` in the condition of the `if` statement.

There are several differences between the string argument supplied to the `RegExp()` constructor and the regular expression given earlier. First, there are more backslash characters (`\`) in the string. This is because, within a `String` literal, the backslash character is normally used to begin an escape code used to represent some Unicode character (Section 4.8). But within a regular expression the backslash is used to represent, not a Unicode character, but a special subexpression within the larger regular expression. Therefore, when a backslash appears in a `String` literal as part of a regular expression, the backslash must be escaped by preceding it with another backslash.

The other difference is that the string argument passed to `RegExp()` begins with a caret (`^`) and ends with a dollar sign (`$`). These are necessary because of the way JavaScript interprets the argument to `RegExp()`. Specifically, although the regular expression `\d\d\d` represents strings *consisting of* three consecutive digits, when it is passed to `RegExp()` it is treated as though it represents all strings *containing* three consecutive digits. The caret and dollar sign characters can be used to override this `RegExp()` interpretation.

Client-Side Programming

In particular, in a JavaScript regular expression, the caret and dollar sign characters represent the beginning and end of a string, respectively. Thus, the argument represents any string in which the beginning of the string is immediately followed by three consecutive digits, the last of which is immediately followed by the end of the string. These characters can be used for other purposes as well. For example, if we removed the dollar sign from the argument to `RegExp` we obtain

```
var acTest = new RegExp("^\\d\\d\\d");
```

which represents the set of all strings that begin with three digits.

JavaScript provides an alternate syntax for creating a `RegExp` instance that avoids the need to duplicate backslashes. For example, the preceding declaration of `acTest` can be rewritten as

```
var acTest = /^\\d\\d\\d/;
```

The expression on the right-hand side of the assignment is known as a *regular expression literal*. The scripting engine conceptually converts such an expression to a call to the `RegExp()` constructor, automatically escaping any backslash characters contained in the regular expression literal. Notice that it is still necessary to use the caret and/or dollar sign characters if you desire to override the `RegExp()` treatment of regular expressions. JavaScript regular expressions are normally written using this literal syntax.

Now that you know something about what regular expressions are and how they can be used in a JavaScript program, the remainder of this section will discuss several regular expression features that are frequently used in browser-based JavaScript programs. Refer to the ECMAScript reference [ECMA-262], particularly Section 15.10, for complete details.

The simplest form of regular expression is a character that is not one of the regular expression special characters, which are

```
^ $ \ . * + ? ( ) [ ] { } |
```

For example, underscore (`_`) is not one of the special characters, so `_` is the regular expression that represents the set of all strings that contain the underscore character. A special character is escaped by preceding it with a backslash. As an example, `\\$` represents the set of strings that end with a dollar sign.

Another simple regular expression is the period (dot) `.`, which represents any character except for a line terminator. In other words, dot is the wildcard character in regular expressions. *Escape code* regular expressions, such as `\\d`, similarly represent multiple characters. Table 4.10 lists all of the JavaScript escape codes representing multiple characters.

Simple regular expressions can be composed into more complex regular expressions using one of three types of operators. We've already seen an example of the first of these operators, *concatenation*. In general, the concatenation of two regular expressions represents the set of strings that consist of any string represented by the first regular expression concatenated with any string represented by the second expression. Thus,

```
^\\d\\. \\w$
```

TABLE 4.10 JavaScript Multicharacter Escape Codes

Escape Code	Characters Represented
<code>\d</code>	Digit: 0 through 9
<code>\D</code>	Any character except those matched by <code>\d</code>
<code>\s</code>	Space: any JavaScript white space or line terminator (space, tab, line feed, etc.)
<code>\S</code>	Any character except those matched by <code>\s</code>
<code>\w</code>	“Word” character: any letter (a through z and A through Z), digit (0 through 9), or underscore (<code>_</code>)
<code>\W</code>	Any character except those matched by <code>\w</code>

represents the set of strings beginning with a single digit followed immediately by a period, a space, and terminating a “word” character (letter, digit, or underscore). Notice that white space is significant within a regular expression. For instance, the set of strings represented by the last regular expression does not include either of the strings

`3.A` or `7.  j`

—the first because it has no space, and the second because it has two spaces.

When we want to concatenate a regular expression with itself multiple times, we can use the *quantifier* shorthand notation. For example, the set of all strings of exactly three digits is represented by the regular expression

`\d{3}`

This notation, viewed as a postfix unary operator, has higher precedence than both concatenation and union (discussed next). Therefore,

`-\d{3}`

represents strings that begin with a `-` followed by three digits.

The second regular expression operator is *union*, which is represented by the pipe symbol `|`. For example,

`\d|\s`

represents the set consisting of all digit and white space characters. The concatenation operator takes precedence over union, so the set of strings represented by the regular expression

`\+|-\d|\s`

consists of `+` (escaped because it is a special character), the two-character strings beginning with `-` followed by a digit, and the white space characters. The default precedence can be overridden using parentheses. Thus,

Client-Side Programming

```
(\\+|-)\\d\\s
```

represents the set of two-character strings beginning with either a + or - followed by a digit plus the white space characters.

It would be tedious (and error prone) to use the union notation directly to represent a set such as the set of all lowercase letters. JavaScript provides a *character class* notation that can be used for such purposes. For instance, the set of lowercase letters can be represented by

```
[a-z]
```

Similarly, the escape code \\w is equivalent to the regular expression

```
[a-zA-Z0-9] | _
```

Notice that, while - is not normally a special character in regular expressions, it is special within a character class and must be escaped by a preceding backslash in order to represent the - character itself.

A variation of the quantifier notation can be used to represent unions of concatenations. For example,

```
\\d{3,6}
```

is shorthand for any string of from three through six digits, that is,

```
\\d\\d\\d|\\d\\d\\d\\d|\\d\\d\\d\\d\\d|\\d\\d\\d\\d\\d\\d
```

By definition, zero concatenations of any string with itself results in the empty string. Thus, the regular expression

```
(\\+|-){0,1}\\d
```

represents the set of strings that begin with either the empty string or a plus or minus sign, followed by a digit. In other words, the leading sign character is optional. JavaScript uses the symbol ? for this purpose. That is to say, the quantifier {0,1} occurs so often that it has its own shorthand symbol, ?. Therefore, the expression displayed would normally be written as

```
(\\+|-)?\\d
```

So far, the regular expressions we've seen have all represented finite sets of strings. The final operator, the *Kleene star*, allows us to represent infinitely large sets. For example, the regular expression

```
\\d*
```

represents the set of strings of any number of digits, including the string of no digits at all (the empty string). In general, the Kleene star following a regular expression represents the set created by concatenating strings from the original set with one another an arbitrary number of times. This postfix unary operator has the highest precedence of all regular expression operators.

For example, assume that a certain application requires the use of passwords and that each password must contain at least one digit and at least one letter and may only contain digits, letters, and underscores. The following is a JavaScript regular expression that represents the set of valid passwords for this application:

```
\w*(\d\w*[a-zA-Z]|[a-zA-Z]\w*\d)\w*
```

4.13 JavaScript Debuggers

At this point, you should know quite a bit about writing JavaScript programs. We'll now briefly learn about debugging JavaScript programs using interactive debugging software. I'll also mention some of the ways in which JavaScript debugging differs from, say, Java debugging.

The first question is whether or not you already have a debugger and, if not, where to obtain one. If you are debugging your JavaScript code using IE6, your system may or may not already have a debugger present. To find out, select **Tools|Internet Options . . .** from the IE6 menu and click on the Advanced tab in the popup window that appears. Make sure that the checkbox for "Disable Script Debugging (Internet Explorer)" is unchecked, and click the OK button in the pop-up. Now load a JavaScript program into IE6 that has an error in it (the file `error.js`, available at the textbook Web site, can be used). If a pop-up window appears asking if you would like to debug, then you have a debugger installed. Otherwise, Microsoft provides a free Script Debugger that is currently available at <http://msdn.microsoft.com/downloads/list/webdev.asp>.

On the other hand, if you are debugging using Mozilla and you downloaded Mozilla according to the instructions in Appendix A, then you automatically have an interactive JavaScript debugger named Venkman. Since Venkman can be run on all major operating systems, I'll use it as a specific example in this section. Other interactive debuggers, such as Microsoft's Script Debugger, generally have similar features, although of course details will differ. You should see the manufacturer's documentation for your debugger for complete details. For Venkman, the documentation can be accessed through the Venkman **Help** menu.

To debug a JavaScript program with the Venkman debugger, begin by starting Mozilla and then opening the debugger window (Figure 4.18) by selecting **Tools|Web Development|JavaScript Debugger** from the Mozilla menu. Next, in the browser window, open the HTML document that loads the JavaScript program to be debugged. The

Loaded Scripts panel in the upper left portion of the debugger window will display the name of your JavaScript file. Right-clicking the file name and selecting Find File from the pop-up context menu then loads the file into the source code panel in the upper right portion of the debugger window.

Figure 4.18 shows the Venkman debugger window after I loaded the `testjs.html` document into Mozilla and then entered `localscope.js` when prompted. `testjs.html`

Client-Side Programming

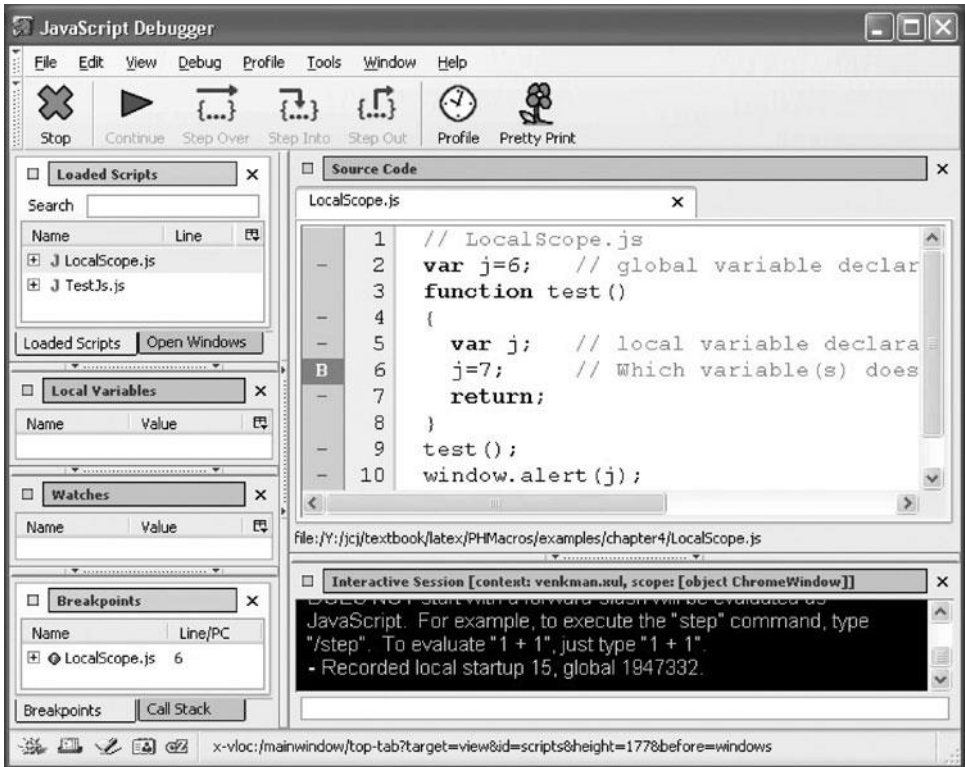


FIGURE 4.18 The Venkman JavaScript debugger with a breakpoint set in a loaded JavaScript file.

loaded `test.js`, which in turn loaded `LocalScope.js`. Thus, the Loaded Scripts panel shows two JavaScript files. I then selected `LocalScope.js` for debugging as just described. Next, I set a *breakpoint* on line 6 of this file by clicking to the left of the line number, which caused a **B** to appear where I clicked. The breakpoint is also listed in the Breakpoints panel at the bottom left of the debugger window.

Now if I return to my browser window, reload it, and again enter `LocalScope.js` when prompted, the scripting engine will *break* (pause) just before executing line 6 of `LocalScope.js`. I can then use the Step buttons to step through the code one line at a time. The Step Over button causes the scripting engine to execute until reaching the next line of code within the current file, so this would take me to line 7 of `LocalScope.js`. The Step Into button behaves similarly unless the current line is a call to a user-defined function or method, in which case the scripting engine enters that function or method and then breaks. Step Out, on the other hand, executes a function or method until it has returned to the calling statement, at which point a break occurs. Clicking Step Out after the break at line 6 of `LocalScope.js` will cause the scripting engine to execute until reaching the end of line 9, the statement that called the `test()` function containing line 6. These Step buttons enable you to carefully examine the flow of control through the program. Alternatively, clicking

on Continue will cause the scripting engine to execute until either another breakpoint or the end of the program is reached.

In addition to control flow, we are often interested in examining the values of variables as a program executes. The Local Variables panel in Venkman provides this information, but in a way that is somewhat confusing due to JavaScript's internal representation of variables. To begin with, I suggest clicking on the small box in the upper left corner of the Local Variables panel; this will float the panel out into a separate window that can be independently resized (Figure 4.19).

As shown in Figure 4.19, the Local Variables panel has just two top-level variables: `scope` and `this`. This particular figure shows the local variables at the time of the break at line 6 of `localScope.js`. When local variables are examined in the midst of executing a function or method, `scope` represents an object that contains the local variables and formal parameters of the function as properties. So the `j` under `scope` represents the local variable declared at line 5 of `localScope.js`. The variable's value is shown as `void` (Venkman's representation of the Undefined value) because line 6 has not yet been executed, so the local `j` has not yet been assigned a value. If we view Local Variables at a point in the execution that is outside all functions and methods, then `scope` will represent the window (global) object. Similarly, except when executing a method, `this` represents the window object. In a method, as mentioned earlier, `this` represents the object on which the method is called.

What about global variables? As indicated earlier, global variables are stored as properties of the window object. Therefore, if either `scope` or `this` represents the window object, then all global variables will appear as properties and can be examined by clicking the `+` to the left of the object name. So, in this example, we would find the global `j` (with value 6) as a property under the `this` object. However, be warned that window is given a great many other properties automatically, so your program's global variables will not be immediately apparent. If neither `scope` nor `this` represents the window object (which is the case if a method is being executed), then the `window` property under the `this` object will represent the global object and can be examined for the values of global variables.

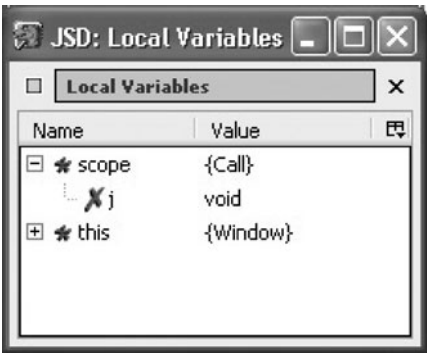


FIGURE 4.19 The Venkman Local Variables panel floated into a separate window.

4.14 References

The primary reference for this chapter, of course, is the ECMAScript 3.0 Language Specification [ECMA-262]. A good starting point for information about Mozilla's implementation of JavaScript is <http://www.mozilla.org/docs/web-developer/#scripting>, while <http://msdn.microsoft.com/library/en-us/script56/html/js56jsoriJScript.asp> provides links to both tutorial and reference information on Microsoft's JScript language.

Be advised that many JavaScript references on the Web are not just about the JavaScript language itself, but also about how JavaScript interacts with web browsers. Such material will probably make more sense once you have read the next chapter.

Exercises

- 4.1. While the value of the Null data type is represented by the JavaScript keyword `null`, the Undefined type has no associated keyword in the JavaScript language. Assume that the variable `testvar` has been declared in a JavaScript program. Write JavaScript code that could be inserted into this program that will output the string `undefined` if `testvar` has the Undefined type's value and output `defined` otherwise.
- 4.2. What is output if the following JavaScript program is executed by a typical web browser? Explain.

```
var i=5;
function duh()
{
    var i;
    window.i=6;
}
duh();
window.alert(i);
```

- 4.3. To what does the following JavaScript expression evaluate? Explain in detail how JavaScript arrives at this result.

```
Number("007") == "007"
```

- 4.4. What is output if the following JavaScript program is executed by a typical web browser? Explain.

```
var o1 = new Object();
o1.j = 9;
var o2 = o1;
function test(o1)
{
    o1.j=10;
    return;
}
test(o1);
window.alert(o2.j);
```

- 4.5. Write JavaScript code that will create an Object with a property named `color` having a String value of `red`.

- 4.6. JavaScript code used by a Web document must be in a file that can be read by anyone who can access the document. In an attempt to keep others from using their code, some JavaScript authors *obfuscate* their code—that is, attempt to reduce the intelligibility of their code—in various ways. What does the following obfuscated code do, and why?

```
var weird = "a1" + "father".slice(4, 6) + "t";  
window[weird]("Weird, but it works.");
```

- 4.7. Insert parentheses in the following expression in order to make the operator precedence relationships clear.

```
a = b ? z /= y ? x : w ? v : u : d += e
```

- 4.8. Some applications, such as random number generators and certain cryptographic algorithms, are explicitly designed to use arithmetic *overflow* as part of their computation. An overflow can occur in Java, for example, when two large `int`'s are multiplied together and the result is stored in an `int`. If the result of the multiplication exceeds 32 bits, only the lower 32 bits are stored (recall that a Java `int` stores 32 bits, with the most significant being the sign bit). The higher-order bits, including the original sign bit, are lost, and the new most-significant bit becomes the new sign bit. Explain how such an overflow can be simulated in JavaScript using one of the JavaScript bit operators.

- 4.9. Consider the following JavaScript program:

```
function addTo(value) {  
    myVar += value;  
    return;  
}  
// Create an object with property myVar  
// and method addTo  
var o = new Object();  
o.myVar = 0;  
o.addTo = addTo;  
// Call the addTo method  
o.addTo(12);
```

This program will throw an exception. Which statement causes the exception? Why? How should this code be corrected?

- 4.10. What is the output of the following JavaScript program? Explain why this output is produced.

```
function rusty(a) {  
    this.x = a;  
    return;  
}  
var o1 = new Object();  
var o2 = new Object();  
o1.rusty = rusty;  
o2.rusty = o1.rusty;  
o1.rusty(1);  
o2.rusty(2);  
window.alert(o1.x + ", " + o2.x);
```



FIGURE 4.20 Output produced by `drawGrid()` when passed `ttt` array as an argument.

- 4.11. Write a JavaScript function `drawGrid()` that takes a two-dimensional array as its sole argument. Your function should produce an alert box that displays the array with grid lines separating the elements. For example, Figure 4.20 shows the output when the function is called with the array `ttt` of Section 4.11.1. While this array is rectangular, your function should work for ragged arrays as well (a two-dimensional array is *ragged* if the one-dimensional arrays that compose it are not all of the same length).
- 4.12. Write a function `median()` that accepts a one-dimensional array containing an odd number of integers as its single argument and returns the median value stored in the array (this is the value of the middle element of a sorted version of the array). The argument array should not be modified.
- 4.13. Using a `RegExp` instance, write a JavaScript function `isValid()` that accepts a `String` argument and returns `true` if the argument matches one of the following phone-number formats and returns `false` otherwise:

```
(123)456-7890
(123) 456-7890
123/456-7890
123-456-7890
123 456 7890
1234567890
```

Research and Exploration

- 4.14. If a JavaScript program attempts to access the value of a variable that does not exist, the scripting engine will throw an exception. For what inputs will the following program throw an exception, and why?

```
var answer = window.prompt("Enter a number", "");
if (answer < 0) {
    isNeg = true;
}
window.alert("Number is negative: " + isNeg);
```

Will the program still throw an exception if the statement

```
var isNeg;
```

is appended following the call to `alert()`? Why or why not? Answer the same questions if instead of appending this statement the body of the `if` statement is changed to

```
var isNeg = true;
```

- 4.15.** When a function is called, it automatically has access to an object named `arguments` that represents the arguments passed to the function. Read about this object in Section 10.1.8 of the ECMAScript reference [ECMA-262], and then write a JavaScript function `summer` that accepts any number of arguments of type `Number` and returns the sum of all the numbers. For example,

```
summer(5, 3, -6, 1)
```

should return the value 3. Calling the function with no arguments—`summer()`—should return 0.

- 4.16.** Use the `toString()` method inherited by `Number` instances to convert a number to hexadecimal. Experiment with your code in the browser(s) assigned by your instructor and report on the format of the string produced, noting details such as the case of letters and how the base of the number is represented, if at all. Also experiment with some other common number bases (such as 2 and 8) as well as at least one uncommon base (such as 24), and report on the results.
- 4.17.** Every user-defined JavaScript function is an instance of the built-in object `Function` and inherits certain properties as a result. One of the inherited properties is a method (function object) named `call()`. Read Section 15.3.4.4 of [ECMA-262] and then explain what the following code does:

```
function simple () {  
    this.count = 10;  
}  
var anObj = new Object();  
simple.call(anObj);
```

(Note: In [ECMA-262] terminology, every `Object` that represents a function has a `[[Call]]` property, which can be thought of as containing the code for the function. Also, `ToObject()` represents an internal JavaScript function that converts its argument to an `Object` as explained in Section 4.12.2.)

- 4.18.** A JavaScript program can direct the scripting engine to execute on a string that is created at run time by calling the built-in `eval()` function. For example,

```
var expr = window.prompt("Enter an arithmetic expression", "");  
window.alert(eval(expr));
```

will prompt the user to enter an arithmetic expression (such as `3+5*2`) and will then output the value of this expression (13 in this example). Perform experiments building on this example in order to answer the following questions:

- (a) Can an expression passed to `eval()` access variables declared elsewhere in the program?

Client-Side Programming

- (b) Can the string passed to `eval()` represent one or more statements rather than simply an expression?
- (c) If a variable is created by the code in the argument to `eval()`, where (in what object) is that variable created? Does your answer change if `eval()` is called from within the body of a function?

4.19. Consider the following JavaScript program:

```
// makeAdd.js
function makeAdd(op1) {
    return function adder (op2) { return op1 + op2; };
}
var add4 = makeAdd(4);
window.alert(add4(6));
```

`makeAdd()` evaluates a function expression for a function named `adder()`, which creates a function object. This function object, in turn, remembers the value of `op1` as it exists at the time `makeAdd()` is called. So, when `makeAdd()` is called with the argument 4, the `adder()` function created remembers the value of `op1` as 4. The object representing this function is returned by `makeAdd()` and assigned to the variable `add4`. When `add4()` is then called as a function with the argument 6, the `adder()` function contained in the `add4` object is actually called, and its `op2` parameter is given the value 6. This function then adds the remembered value of `op1` to the argument value associated with `op2`, returning the value 10. In general, an object that combines a function with values remembered from the scope in effect at the time the function was created is called a *closure*. Thus, JavaScript function expressions actually create closures, not merely functions. Beginning with the given program, experiment with closures in JavaScript, addressing the following questions:

- (a) What happens if you call `add4()` with a value other than 6?
- (b) Add code that calls `makeAdd()` a second time with a different argument value and assigns the result to a second variable. Does your new closure behave as you would expect? Does the original closure (`add4()`) still behave as it did before, or is it affected by the creation or execution of a second closure? Explain your observations.
- (c) Add a global variable `op3` to the program, and change the body of the function expression to

```
return op1 + op2 + op3;
```

Does changing the value of `op3` after the call `makeAdd(4)` change the value returned by a subsequent call to `add4()`? Explain.

Projects

- 4.20.** Write portions of a *discrete event simulation* in JavaScript. Discrete event simulations are often used to analyze customer service operations, such as an airline ticket counter. An *event* in such a simulation represents something of note that happens, such as a customer arriving at the counter or a customer transaction being completed. The heart of the simulation is an *event queue*, which is a time-ordered list of upcoming events. In addition to the time at which the event will occur, each element of the event queue has a field indicating the type of event (such as “transaction complete”). The program iterates through the event queue, each time removing and processing the event at the front of the

queue. This may cause other events to be added to the queue (they will not necessarily be added to the end, but instead are added so that the event times remain ordered; so this is actually a priority queue). This iteration continues until the queue is exhausted, at which time information gathered during program execution (such as the average time customers spent waiting in line for an agent) is output.

- (a) Write a constructor for a class `Event`; the instances of this class will be used to represent simulation events. Each instance should have properties `time` (assume that this is a `Number`) and `type` (of arbitrary type). Your constructor should have two arguments corresponding to these properties, so that the properties can be initialized easily when an instance is constructed.
- (b) Write a constructor and methods for a class `EventQueue` representing an event queue. There should be a method `empty()` that returns `true` if and only if the queue is empty; a method `add()` that takes an `Event` object as its argument and adds it to the queue; and a method `removeNextEvent()` that removes the `Event` with the smallest time value (breaking ties arbitrarily) from the event queue and returns this object.
- (c) Use the classes of (a) and (b) to write a simple airline ticket counter simulation. Input the number of ticket agents working at the counter. The event queue should be initialized by adding a number of customer-arrival events at various times, each time being an integer representing the number of minutes since the ticket counter opened. The program should then begin processing the event queue by removing the first event. If no agent is free when a customer-arrival event occurs, an element containing the current time should be pushed onto a data structure representing the waiting line at the ticket counter (so each element of the line structure represents a customer by the time at which the customer entered the line). If an agent is free when a customer-arrival event occurs, then the number of busy agents is incremented and a transaction-complete event is scheduled to occur three minutes later. When a transaction-complete event occurs, if any customers are in the waiting-line structure, then the first customer is removed from the line and the length of time that customer waited in line is added to an accumulator variable; the number of busy agents is unchanged. Otherwise, if no customers are waiting, an agent becomes free. The simulation ends when no events remain in the event queue. At this time, average customer wait time (accumulated wait time divided by number of customers) is output.

- 4.21. Write a JavaScript program that will allow a user to test the speed at which alert boxes are generated by a particular browser running on a particular system. Specifically, the program will display an alert box, then display a second alert box after OK is clicked in the first alert box, display a third alert box when the second is acknowledged, and so on. The program will stop displaying these alert boxes once 10 seconds has elapsed from the time the first box was displayed (so dividing by 10 will give the number of boxes displayed per second). Then, for 2 seconds, the program should repeatedly display alert boxes that report on the number of clicks per second recorded during the first 10 seconds (the reason for doing this for 2 seconds is so that you can see the results even if you accidentally click OK on one or more of the result alert boxes). For timing, use only the built-in `Date` object and its methods; that is, don't use any other timing-related host objects that might be available in your browser. Report on the results of testing with various browsers and systems.
- 4.22. Write a JavaScript program that generates 10 random math problems. Each math problem should consist of three operands (random integers between -10 and 10, inclusive) and two operators (randomly drawn from +, -, *, and %). For example, a problem might be `-4 + 7 % 3`. Display each problem in a prompt box, and allow the user to enter an integer

Client-Side Programming

representing the value of the problem (following standard JavaScript operator precedence rules). After all 10 problems have been answered, output the number of correct answers. (Hint: Use `eval()` (Exercise 4.18) to compute the correct answers.)

- 4.23.** Consider a two-player game played with pegs of six different colors. One player (the *encrypter*) selects four pegs of different colors and places them in a certain order (leftmost through rightmost). This is the *encrypted message*, and is not shown to the second player (the *decrypter*). The decrypter then selects four pegs and places them in an order from left to right; this is the decrypter's guess about what the encrypted message is. Next, the encrypter *leaks* information about the encrypted message by placing small black or white pegs next to the guessed pegs; these pegs define a *score* for the guess. A black scoring peg indicates that one of the guessed pegs is exactly right: right color and in the right position. A white scoring peg indicates that a guessed peg is the right color but in the wrong position. So two black scoring pegs and one white would mean that three of the guessed pegs are of the right color and that two of these are in the correct position within the guessed peg ordering. Once a guess has been scored, the decrypter guesses a second time, and the encrypter scores this guess. (See Figure 5.35 for a graphical representation of such a game after two guesses have been scored.) This continues until the decrypter fully guesses the secret code (score of four black pegs).

Write a JavaScript program that plays a text version of this game. The program should simulate the encrypter, and the human player will be the decrypter. Specifically, the program will randomly select a secret code consisting of four numbers between 1 and 6. The numbers should all be different. The program will then input and score each guess made by the decrypter (human player). In particular, the user will input four numbers in a prompt box, and the computer will score this guess against the secret. The computer will then output to an alert box the user's guess and score (so many black and so many white pegs). If the score is 4 black, the game ends and the computer displays the total number of guesses made. Otherwise, the computer asks for another guess consisting of four numbers. Notice that the human is expected to write down his or her guesses and scores, because the computer will only output the most recent guess and score by the user each time. Your program does not need to verify that the user enters numbers between 1 and 6, since entering something else can only hurt the user's chances of winning. You may also assume that all of the user's four inputs for a guess are different numbers.

- 4.24.** Write two versions of a `base2()` function that takes a String argument and returns a Boolean value indicating whether or not the argument begins with a 1 followed by any number of 0 or 1 characters and ending with the string `_2`. For example, it should return `true` if called with `10010_2` but `false` with `0_2`. One version should use regular expressions, and the other should not.
- 4.25.** Consider a word game in which a 4-by-4 grid of letters is displayed. Your task is to find as many words as possible in the grid. A word occurs in the grid if, starting at a position in the grid that contains the first letter of the word, the second letter is in a grid position next to (including diagonally) the first, the third letter is in a position next to the second, and so on. No grid position can be used twice in making a word. So, if the grid of letters is

```
e|w|t|h
-----
r|d|v|x
-----
q|p|a|w
-----
z|y|b|m
```

then it contains words such as “paw” and “red” but does not contain “ewe” because the upper-left corner of the grid would have to be used twice to make this word.

- (a) Write a program that allows a user to play a solitaire, three-letter version of the game described. Your program should incorporate a dictionary of all (or at least most) three-letter words; a JavaScript file `shortwords.js` containing over 400 such English words is included as part of the download available from the textbook supplements Web site (see the Preface for the URL). The program should begin by selecting 16 letters at random. Display these letters as a 4-by-4 grid in an alert box (this probably won’t look perfect because of the fonts used in alert boxes). When the user clicks OK in this box, a prompt box should be displayed in which the user is asked to enter a word. If the word entered is in the dictionary and in the grid and hasn’t been entered before, the program should increment a counter. Next, the alert box with the grid should again be displayed, followed by the prompt box and a check of the word entered. This should continue until the user enters the word `quit`. The program should then display a final alert box showing the grid and the number of distinct words found by the user.
- (b) Modify your program so that it displays a message at the beginning of each alert box after the first. The message should tell the user that the previously entered word was a duplicate, was not on the board or in the dictionary, or was accepted.
- (c) Modify your program so that it will be more likely to choose some letters (such as vowels) than it is other letters.
- (d) Modify your program so that when it terminates it displays a list of all of the words not found, along with the other information.
- (e) Modify your program so that it automatically terminates once 3 minutes have elapsed from the time the first alert box was displayed. That is, clicking OK in either the alert or prompt box after 3 minutes have elapsed should cause the program to terminate and the final alert box to be displayed. The word entered in a final prompt box (one submitted after 3 minutes have elapsed) should not be counted.

4.26. Crazy Eights is a simple game played with a standard deck of 52 playing cards. The rules for a two-player version are as follows: The deck is shuffled, seven cards are dealt to each player, one card is placed face up to begin a discard pile, and the remaining cards are placed face down and form a draw pile. The first player can play one card from his or her hand face up on the discard pile if that card matches the suit or the number of the card currently on the discard pile. Alternatively, if the player holds an 8 (any suit), this card can be played, and the player also names the suit that this card represents. For example, if the top discard card is a 4 of spades, I can play an 8 of hearts and specify that the suit is now clubs. The next card played to the discard pile by either player must either be an 8 or a club. If the first player does not play, then he or she must take one card from the top of the draw pile. The second player then plays similarly. Play alternates until one of the players is out of cards; this player wins.

- (a) Write a JavaScript program (and an HTML document that loads the program and any other necessary files) that allows a human to play Crazy Eights against a computer. Specifically, the program should simulate the initial shuffle and dealing of cards by creating appropriate data structures. It should next display a prompt box telling the user what cards he or she holds and what card is on top of the discard pile (prompt boxes in IE6 display only two lines, so you should use short card names such as 6c and kh). The user should then enter in the prompt box either a p indicating that a card should be picked from the draw pile, or a string (such as 6c) representing a card to be played. If the card is not legal (either not in the hand or not of the

Client-Side Programming

correct suit/number), prompt the user again. After simulating the user's play, play the computer's hand, always playing a card if possible. Alternate allowing the user to play and playing the computer's hand until one of the players wins, at which time the program should announce the winner.

For this version of the program, if an 8 is played, assume that the suit of the card played is the suit named by the player. For example, if the computer plays the 8 of diamonds, the suit will be diamonds.

Shuffling can be performed as follows: if there are n cards, choose a random number r_1 between 0 and $n - 1$. Swap the card at index r_1 with the card at index $n - 1$. Then choose a random number r_2 between 0 and $n - 2$, and swap the card at r_2 with the one at $n - 2$. Continue this process $n - 1$ times. The JavaScript code

```
Math.floor(Math.random()*(n+1))
```

evaluates to a random number between 0 and n .

- (b) If the game continues for a long time, the draw pile may become empty. Modify your program so that if this happens, a new draw pile is created by shuffling the cards underneath the top card of the discard pile. Following this operation, the discard pile will consist of just the single card that was originally on top of the pile.
- (c) Modify the program so that the user can name the suit after playing an 8. That is, if the user plays an 8, prompt the user a second time for the suit it is to represent. Be careful: you should not actually change the suit of the 8 played, since the discard pile may be shuffled later to form a new draw pile. Instead, the player (and your program) is supposed to remember what suit was named.
- (d) Write and use in your program a JavaScript class named `cards` that can be used to represent the shuffled deck, the players' hands, and the discard pile. This class should have at a minimum methods for adding individual cards to and removing cards from an instance, "looking at" cards (for example, obtaining a reference to the third card in a hand), and shuffling the cards in an instance.
- (e) Add a `sort` method to the `cards` instance. This will be used to sort an instance representing a player's hand, so that the cards appear in a logical order. For example, the primary sort might be by suit, with a secondary sort placing cards in ascending order of their number (2 through 10 followed by jack, queen, king, and ace).
- (f) Write and use in your program a JavaScript class named `card` that can be used to represent individual cards. The constructor for this class should take two arguments representing the suit and number of a card. Write at least two methods for the class: one that returns a string representing the card suitable for output to the user (such as 6c), and the other that returns a `Number` representing the *sort order* of the card. This is a value that can be used to sort the cards appropriately. For example, the 2 of clubs might be given a sort order of 0, the 3 of clubs 1, the ace of clubs 12, the 2 of diamonds 13, and so on.